CP352101
Data Science Textbook
อ.ธนพล ตั้งชูพงศ์
Dark Mode

หน้าหลัก
1 Data Science คืออะไร?
2 การตั้งคำถามและกำหนดปัญหา
3 Python สำหรับ Data Science
4 การเก็บข้อมูลและ Pandas
5 การทำความสะอาดและแปลงข้อมูล
6 สถิติเชิงพรรณนา EDA และ Data Visual
7 แนะนำ Machine Learning
  Machine Learning คืออะไร?
  ประเภท Machine Learning
  ML Workflow
  Bias-Variance Tradeoff
  scikit-learn API
  LLM ในกระบวนการ Data Science
  สรุปท้ายบทที่ 7
  เอกสารอ้างอิงประจำบทที่ 7
  แบบฝึกหัดท้ายบทที่ 7
8 Regression — การทำนายค่าต่อเนื่อง
9 Classification — การจำแนกประเภท
10 Ensemble Methods
11 Clustering — การจัดกลุ่มข้อมูล
12 Model Validation และ Selection
13 Data Storytelling และการเขียนรายงาน
14 โครงงาน Data Science: End-to-End Pr
อภิธานศัพท์
ดรรชนี

บทที่ 7: แนะนำ Machine Learning

ภาค 3: Machine Learning CLO: CLO1, CLO3 **สัปดาห์:** 9

วัตถุประสงค์การเรียนรู้

เมื่อศึกษาบทนี้จบแล้ว นักศึกษาจะสามารถ:

1. อธิบายความหมายของ Machine Learning และความสัมพันธ์กับ AI ได้
2. แยกแยะประเภท ML (Supervised, Unsupervised, Reinforcement) ได้
3. อธิบาย ML Workflow ตั้งแต่เตรียมข้อมูลถึง Evaluation ได้
4. อธิบายแนวคิด Bias-Variance Tradeoff ได้
5. ใช้ scikit-learn API พื้นฐาน (fit, predict, score) ได้
6. อธิบายบทบาทของ LLM ในกระบวนการ Data Science ได้

Machine Learning คืออะไร?

Machine Learning (ML) หรือการเรียนรู้ของเครื่อง เป็นแขนงหนึ่งของวิทยาการคอมพิวเตอร์และปัญญาประดิษฐ์ (Artificial Intelligence) ที่มุ่งเน้นการพัฒนาอัลกอริทึมและแบบจำลองทางสถิติ เพื่อให้ระบบคอมพิวเตอร์สามารถเรียนรู้และปรับปรุงประสิทธิภาพการทำงานจากข้อมูล (Data) ได้โดยอัตโนมัติ โดยไม่ต้องอาศัยการเขียนโปรแกรมเพื่อสั่งการอย่างเฉพาะเจาะจง (Explicitly programmed) ในทุกขั้นตอน (James et al., 2023)

แนวคิดพื้นฐานของ Machine Learning ได้รับการนิยามอย่างเป็นทางการโดย Tom Mitchell ในปี ค.ศ. 1997 ซึ่งเป็นนิยามที่ได้รับการยอมรับอย่างกว้างขวางในแวดวงวิชาการดังนี้:

> *"A computer program is said to learn from experience* ***E*** *with respect to some class of tasks* ***T*** *and performance measure* ***P****, if its performance at tasks in* ***T****, as measured by* ***P****, improves with experience* ***E****."*

พิจารณาตัวอย่างการประยุกต์ใช้นิยามดังกล่าวกับ **ระบบคัดกรองอีเมลขยะ (Spam Email Filter)**:

- **Task (T):** การจำแนกอีเมลที่รับเข้ามาว่าเป็น "อีเมลขยะ (Spam)" หรือ "อีเมลปกติ (Ham)"
- **Experience (E):** ฐานข้อมูลอีเมลในอดีตที่ผู้ใช้งานเคยกดรายงานว่าเป็นอีเมลขยะ รวมถึงอีเมลปกติที่ผู้ใช้เปิดอ่าน
- **Performance (P):** สัดส่วนความถูกต้อง (Accuracy) หรือจำนวนอีเมลขยะที่ระบบสามารถดักจับได้สำเร็จโดยไม่บล็อกอีเมลสำคัญผิดพลาด

จากการศึกษาพบว่า เมื่อระบบได้รับข้อมูลอีเมลใหม่ๆ (E เพิ่มขึ้น) อัลกอริทึมจะทำการปรับปรุงพารามิเตอร์ภายใน ทำให้ความแม่นยำในการคัดกรอง (P) สูงขึ้นในการทำงานจำแนกอีเมล (T) ในอนาคต

ความแตกต่างระหว่าง Traditional Programming และ Machine Learning

เพื่อทำความเข้าใจกระบวนทัศน์ (Paradigm) ของ Machine Learning อย่างถ่องแท้ นักศึกษาจำเป็นต้องเปรียบเทียบกับวิธีการเขียนโปรแกรมแบบดั้งเดิม (Traditional Programming) ที่นักศึกษาคุ้นเคยในรายวิชาพื้นฐานการเขียนโปรแกรม (Grus, 2019)

1. Traditional Programming (การเขียนโปรแกรมแบบดั้งเดิม): ในกระบวนทัศน์นี้ วิศวกรซอฟต์แวร์จะเป็นผู้กำหนด **กฎเกณฑ์ (Rules)** อย่างชัดเจนผ่านโครงสร้างควบคุม เช่น `if-else`, `for-loop` หรือ `switch-case` จากนั้นนำกฎเกณฑ์เหล่านี้ไปประมวลผลร่วมกับ **ข้อมูลนำเข้า (Data)** เพื่อให้ได้ **ผลลัพธ์ (Answers/Output)**

2. Machine Learning (การเรียนรู้ของเครื่อง): กระบวนทัศน์นี้จะกลับทิศทางกัน ระบบจะรับ **ข้อมูลนำเข้า (Data)** และ **ผลลัพธ์ที่ต้องการ (Answers/Labels)** เข้าไปในอัลกอริทึม เพื่อให้คอมพิวเตอร์ค้นหาและสกัด **กฎเกณฑ์ (Rules) หรือ รูปแบบ (Patterns)** ออกมา กฎที่ได้นี้จะถูกเก็บในรูปแบบของ "แบบจำลอง (Model)" ซึ่งสามารถนำไปใช้ทำนายผลลัพธ์ของข้อมูลชุดใหม่ในอนาคตได้

คุณลักษณะ	Traditional Programming	Machine Learning
สิ่งที่ป้อนเข้าสู่ระบบ (Input)	ข้อมูล (Data) + กฎเกณฑ์ (Rules/Code)	ข้อมูล (Data) + ผลลัพธ์เฉลย (Answers/Labels)
สิ่งที่ระบบสร้างขึ้น (Output)	ผลลัพธ์ (Answers)	กฎเกณฑ์ (Rules/Model)
ความเหมาะสมในการใช้งาน	ปัญหาที่มีตรรกะชัดเจน สามารถเขียนเป็น Flowchart ได้ เช่น การคำนวณภาษี, ระบบจัดการฐานข้อมูล	ปัญหาที่มีความซับซ้อนสูง กฎเกณฑ์มีข้อยกเว้นมาก เช่น การรู้จำใบหน้า, การแปลภาษา
การปรับตัว (Adaptability)	ต้องแก้ไขโค้ด (Hardcode) ด้วยตนเองเมื่อเงื่อนไขเปลี่ยน	โมเดลสามารถเรียนรู้ใหม่ (Retrain) ได้เมื่อมีข้อมูลใหม่

แผนภาพเปรียบเทียบกระบวนทัศน์:

```
[Traditional Programming]
Data  ------\
             +---> [ Computer / Program ] ---> Answers
Rules ------/

[Machine Learning]
Data    ----\
             +---> [ Machine Learning Algorithm ] ---> Rules (Model)
Answers ----/
```

(ดัดแปลงจาก VanderPlas, 2016)

ตัวอย่างโค้ดเปรียบเทียบด้วย Python: พิจารณาระบบตรวจจับข้อความ SMS หลอกลวง (Scam) ในประเทศไทย เช่น ข้อความชักชวนเล่นการพนันหรือให้สินเชื่อ

```python
# ---------------------------------------------------------
# วิธีที่ 1: Traditional Programming (Rule-based approach)
# ---------------------------------------------------------
def is_scam_traditional(sms_text):
    # โปรแกรมเมอร์ต้องกำหนดคำค้นหา (Keywords) ด้วยตนเอง
    scam_keywords = ['สินเชื่อ', 'คลิก', 'อนุมัติไว', 'แจกฟรี', 'เครดิตฟรี']
```

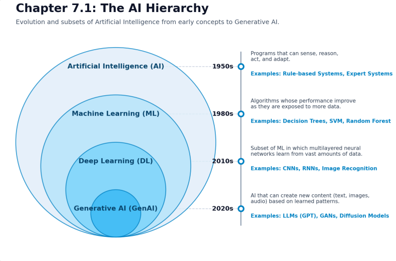
```
    for word in scam_keywords:
        if word in sms_text:
            return True # พบคำต้องสงสัย ถือว่าเป็น Scam
    return False

# ทดสอบ
sms1 = "คุณได้รับสิทธิ์ สินเชื่อ ดอกเบี้ยต่ำ คลิก bit.ly/spam"
print(f"Traditional Prediction: {is_scam_traditional(sms1)}")
# ข้อเสีย: หากผู้ไม่หวังดีเปลี่ยนคำเป็น "กู้เงินด่วน" ระบบนี้จะตรวจไม่พบทันที

# ----------------------------------------------------------
# วิธีที่ 2: Machine Learning Approach
# ----------------------------------------------------------
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# 1. เตรียมข้อมูล (Data) และ ผลลัพธ์ (Answers)
sms_data = [
    "คุณได้รับสิทธิ์ สินเชื่อ ดอกเบี้ยต่ำ คลิก bit.ly/spam", # Scam
    "พรุ่งนี้มีประชุมที่ตึกวิทยบริการ เวลา 10.00 น.",         # Normal
    "แจกฟรี เครดิตฟรี 500 บาท กดรับเลย",                  # Scam
    "อาจารย์ครับ ผมขอส่งงานวิชา Data Science ย้อนหลังครับ"    # Normal
]
# 1 = Scam, 0 = Normal
labels = [1, 0, 1, 0]

# 2. แปลงข้อความให้อยู่ในรูปแบบตัวเลข (Vectorization)
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(sms_data)

# 3. ให้ Algorithm เรียนรู้เพื่อสร้าง Model (Rules)
ml_model = MultinomialNB()
ml_model.fit(X, labels) # กระบวนการเรียนรู้ (Training)

# 4. นำ Model ไปทำนายข้อมูลใหม่
new_sms = ["ด่วน! กู้เงิน อนุมัติไว ไม่ต้องค้ำประกัน"]
new_X = vectorizer.transform(new_sms)
prediction = ml_model.predict(new_X)

print(f"ML Prediction (1=Scam, 0=Normal): {prediction[0]}")
# โมเดลจะคำนวณความน่าจะเป็นจากสถิติของคำที่ปรากฏร่วมกัน โดยไม่ต้องเขียน if-else
```

ผลลัพธ์:

```
Traditional Prediction: True
ML Prediction (1=Scam, 0=Normal): 1
```

ความสัมพันธ์ระหว่าง AI, ML, DL และ GenAI

ในปัจจุบัน คำศัพท์ทางด้านวิทยาการข้อมูลมักถูกใช้สลับกันจนเกิดความสับสน เพื่อให้การสื่อสารทางวิชาการมีความถูกต้อง นักศึกษาจำเป็นต้องเข้าใจลำดับชั้น (Hierarchy) ของเทคโนโลยีเหล่านี้ ซึ่งมีความสัมพันธ์ในลักษณะของเซตย่อย (Subset) ดังแผนภาพด้านล่าง


Chapter 7.1: The AI Hierarchy
Evolution and subsets of Artificial Intelligence from early concepts to Generative AI.
Artificial Intelligence (AI)
Machine Learning (ML)
Deep Learning (DL)
Generative AI (GenAI)
1950s
1980s
2010s
2020s

Programs that can sense, reason, act, and adapt.

Examples: Rule-based Systems, Expert Systems

Algorithms whose performance improve as they are exposed to more data.

Examples: Decision Trees, SVM, Random Forest

Subset of ML in which multilayered neural networks learn from vast amounts of data.

Examples: CNNs, RNNs, Image Recognition

AI that can create new content (text, images, audio) based on learned patterns.

Examples: LLMs (GPT), GANs, Diffusion Models

AI > ML > DL > GenAI Hierarchy

```
+--------------------------------------------------------+

|  Artificial Intelligence (AI)                          |
|  ปัญญาประดิษฐ์: การทำให้คอมพิวเตอร์มีความคิดคล้ายมนุษย์      |
|                                                        |
|  +--------------------------------------------------+  |
|  |  Machine Learning (ML)                            |  |
|  |  การเรียนรู้ของเครื่อง: การใช้สถิติเพื่อให้ระบบเรียนรู้   |  |
|  |  จากข้อมูลได้เอง (เช่น Decision Trees, SVM)         |  |
|  |                                                  |  |
|  |  +--------------------------------------------+  |  |
|  |  |  Deep Learning (DL)                         |  |  |
|  |  |  การเรียนรู้เชิงลึก: ใช้โครงข่ายประสาทเทียม      |  |  |
|  |  |  (Neural Networks) ที่มีหลายชั้น (Layers)    |  |  |
|  |  |                                            |  |  |
|  |  |  +--------------------------------------+  |  |  |
|  |  |  | Generative AI (GenAI)                |  |  |  |
|  |  |  | AI เชิงรู้สร้าง: โมเดลที่สามารถสร้าง       |  |  |  |
|  |  |  | ข้อมูลใหม่ได้ (เช่น ChatGPT, Midjourney)|  |  |  |
|  |  |  +--------------------------------------+  |  |  |
|  |  +--------------------------------------------+  |  |
|  +--------------------------------------------------+  |
+--------------------------------------------------------+
```

1. Artificial Intelligence (AI): เป็นร่มใหญ่ที่สุด (The broadest concept) ครอบคลุมทุกศาสตร์ที่พยายามสร้างเครื่องจักรให้มีความสามารถในการรับรู้ ให้เหตุผล และตัดสินใจได้เทียบเท่ามนุษย์ AI อาจสร้างขึ้นจาก Traditional Programming แบบซับซ้อน (เช่น AI ในเกมหมากรุกยุคแรกที่ใช้เทคนิค Minimax Algorithm) หรือใช้ข้อมูลในการเรียนรู้ก็ได้

2. Machine Learning (ML): เป็นเซตย่อยของ AI ที่เน้นการใช้คณิตศาสตร์และสถิติเป็นแกนหลัก อัลกอริทึม ML จะไม่ถูกตั้งโปรแกรมแบบตายตัว แต่จะ “เรียนรู้” รูปแบบจากชุดข้อมูล (Dataset) เพื่อใช้ในการทำนาย (Prediction) หรือจำแนกประเภท (Classification)

3. Deep Learning (DL): เป็นเซตย่อยของ ML ที่ใช้สถาปัตยกรรมแบบ **โครงข่ายประสาทเทียม (Artificial Neural Networks - ANNs)** ที่ได้รับแรงบันดาลใจจากการทำงานของสมองมนุษย์ คำว่า “Deep” หมายถึงการมีชั้นของโครงข่าย (Hidden Layers) จำนวนมาก DL มีประสิทธิภาพสูงมากในการประมวลผลข้อมูลที่ไม่มีโครงสร้าง (Unstructured Data) เช่น รูปภาพ เสียง และข้อความธรรมชาติ

4. Generative AI (GenAI): เป็นวิวัฒนาการล่าสุดที่ต่อยอดมาจาก Deep Learning โดยเฉพาะโมเดลภาษาขนาดใหญ่ (Large Language Models - LLMs) ในขณะที่ ML แบบดั้งเดิมมักใช้ทำหน้าที่ “วิเคราะห์และทำนาย” (Discriminative) ว่ารูปนี้คือแมวหรือสุนัข แต่ GenAI จะมุ่งเน้นการ “สร้าง” (Generative) ข้อมูลใหม่ที่ไม่เคยมีมาก่อน โดยการเรียนรู้การกระจายตัวของความน่าจะเป็น (Probability distribution) ของข้อมูลต้นฉบับ

การประยุกต์ใช้ Machine Learning ในชีวิตประจำวันและบริบทประเทศไทย

เทคโนโลยี Machine Learning ไม่ได้เป็นเพียงทฤษฎีในห้องปฏิบัติการ แต่ได้ถูกบูรณาการเข้ากับระบบโครงสร้างพื้นฐานและแอปพลิเคชันที่เราใช้งานในชีวิตประจำวัน (Knaflic, 2015) เพื่อให้เห็นภาพชัดเจนขึ้น พิจารณาตัวอย่างการประยุกต์ใช้ในบริบทของประเทศไทยดังต่อไปนี้:

1. ระบบแนะนำสินค้าและบริการ (Recommendation Systems)

- **บริบท:** แพลตฟอร์ม E-commerce เช่น Shopee หรือ Lazada รวมถึงระบบสตรีมมิ่งอย่าง Netflix
- **การทำงานของ ML:** ระบบจะใช้เทคนิคเช่น Collaborative Filtering เพื่อวิเคราะห์พฤติกรรมการคลิก การหยิบใส่ตะกร้า และประวัติการซื้อของนักศึกษา หากนักศึกษาสาขาวิทยาการคอมพิวเตอร์มักจะซื้อหนังสือ “Python Data Science” ควบคู่กับ “คีย์บอร์ด Mechanical” ระบบจะเรียนรู้ความสัมพันธ์นี้และนำไปแนะนำให้นักศึกษาคนอื่นที่มีพฤติกรรมคล้ายคลึงกัน
- **ผลลัพธ์:** เพิ่มยอดขาย (Conversion rate) และสร้างประสบการณ์ที่ดีให้ผู้ใช้งานโดยไม่ต้องค้นหาสินค้าเอง

2. การตรวจจับธุรกรรมทางการเงินที่ผิดปกติ (Fraud Detection)

- **บริบท:** ธนาคารพาณิชย์ในประเทศไทย (เช่น K PLUS, SCB EASY) และระบบพร้อมเพย์ (PromptPay) เผชิญกับปัญหาภัยไซเบอร์และ “บัญชีม้า”
- **การทำงานของ ML:** ธนาคารใช้โมเดลประเภท Anomaly Detection เพื่อเฝ้าระวังธุรกรรมแบบเรียลไทม์ (Real-time monitoring) โมเดลจะเรียนรู้พฤติกรรมปกติของลูกค้า เช่น นาย A มักโอนเงินหลักพันบาทในเขตจังหวัดขอนแก่น หากจู่ๆ มีการโอนเงินหลักแสนบาทไปยังบัญชีปลายทางที่น่าสงสัยในต่างประเทศในช่วงเวลา 03.00 น. โมเดลจะคำนวณค่าความเสี่ยง (Risk Score) และระงับธุรกรรมนั้นชั่วคราวเพื่อให้พนักงานตรวจสอบ
- **ผลลัพธ์:** ลดความเสียหายทางการเงินและป้องกันอาชญากรรมทางเทคโนโลยี

3. ทางด้านสาธารณสุขและการแพทย์ (Healthcare)

- **บริบท:** โรงพยาบาลศรีนครินทร์ มหาวิทยาลัยขอนแก่น และคณะแพทยศาสตร์
- **การทำงานของ ML:** มีการนำอัลกอริทึม Deep Learning (เช่น Convolutional Neural Networks - CNNs) มาใช้วิเคราะห์ภาพถ่ายจอประสาทตา (Retinal images) เพื่อคัดกรองผู้ป่วยโรคเบาหวานขึ้นตา (Diabetic Retinopathy) โมเดลสามารถวิเคราะห์จุดเลือดออกหรือความผิดปกติของเส้นเลือดฝอยในภาพได้อย่างแม่นยำเทียบเท่าจักษุแพทย์
- **ผลลัพธ์:** ช่วยลดภาระงานของแพทย์ และเพิ่มโอกาสในการเข้าถึงการคัดกรองโรคของผู้ป่วยในพื้นที่ห่างไกลภาคตะวันออกเฉียงเหนือ

4. ระบบประมวลผลภาษาธรรมชาติ (Natural Language Processing - NLP)

- **บริบท:** ระบบ Customer Service ของบริษัทโทรคมนาคม (เช่น AIS Aunjai, True iService)
- **การทำงานของ ML:** แชทบอท (Chatbots) ใช้ ML ในการทำความเข้าใจเจตนา (Intent Classification) ของลูกค้าที่พิมพ์ข้อความภาษาไทยเข้ามา ซึ่งภาษาไทยมีความท้าทายสูงในการตัดคำ (Word Tokenization) และการจัดการกับคำสแลง โมเดลจะจำแนกว่าลูกค้าต้องการเช็คยอด แจ้งปัญหาอินเทอร์เน็ต หรือสมัครแพ็กเกจ และทำการตอบกลับอัตโนมัติ
- **ผลลัพธ์:** ให้บริการลูกค้าได้ตลอด 24 ชั่วโมงและลดต้นทุนในการจ้างพนักงาน Call Center

สรุปประเด็นสำคัญ (Key Takeaways)

- **Machine Learning (ML)** คือศาสตร์ที่ทำให้คอมพิวเตอร์สามารถเรียนรู้และพัฒนาประสิทธิภาพ (P) ในการทำงาน (T) จากประสบการณ์หรือข้อมูล (E) ได้โดยอัตโนมัติ
- ความแตกต่างหลักระหว่าง **Traditional Programming** และ **ML** คือ Traditional Programming รับกฎเกณฑ์เพื่อสร้างผลลัพธ์ แต่ ML รับข้อมูลและผลลัพธ์เฉลยเพื่อสร้างกฎเกณฑ์ (Model)
- ลำดับชั้นของเทคโนโลยีคือ **AI** ครอบคลุม **ML**, ML ครอบคลุม **Deep Learning**, และ **Generative AI** เป็นเทคโนโลยีการสร้างข้อมูลใหม่ที่ต่อยอดมาจาก Deep Learning
- ML ถูกนำมาประยุกต์ใช้อย่างกว้างขวางในประเทศไทย ทั้งในด้าน E-commerce (ระบบแนะนำ), การเงิน (การตรวจจับทุจริต), และการแพทย์ (การวิเคราะห์ภาพถ่ายทางการแพทย์)

คำถามท้าย Section

1. ให้นักศึกษายกตัวอย่างระบบ Machine Learning ที่นักศึกษาใช้งานในชีวิตประจำวัน (นอกเหนือจากที่ระบุในเนื้อหา) พร้อมระบุว่าในระบบนั้น Task (T), Experience (E), และ Performance (P) คืออะไร?
2. หากธนาคารแห่งหนึ่งต้องการสร้างระบบอนุมัติสินเชื่ออัตโนมัติ นักศึกษาคิดว่าการใช้ Traditional Programming (ใช้ if-else กำหนดเกณฑ์เงินเดือน) กับการใช้ Machine Learning (เรียนรู้จากประวัติลูกค้าในอดีต) มีข้อดีและข้อเสียแตกต่างกันอย่างไรในบริบทนี้?
3. อธิบายความแตกต่างที่สำคัญระหว่าง Machine Learning แบบดั้งเดิม (Traditional ML) และ Generative AI พร้อมยกตัวอย่างแอปพลิเคชันประกอบ

ประเภท Machine Learning

การแบ่งประเภทของอัลกอริทึมใน Machine Learning มักพิจารณาจากรูปแบบของข้อมูลที่นำมาใช้สอน (Training Data) และวิธีการที่ระบบเรียนรู้จากข้อมูลเหล่านั้น (James et al., 2023) ในทางวิทยาการคอมพิวเตอร์และวิทยาการข้อมูล เราสามารถจำแนก Machine Learning ออกเป็น 4 ประเภทหลัก ได้แก่ Supervised Learning, Unsupervised Learning, Semi-supervised Learning และ Reinforcement Learning ซึ่งแต่ละประเภทมีความเหมาะสมกับลักษณะปัญหาและบริบทของข้อมูลที่แตกต่างกัน

Supervised Learning (การเรียนรู้แบบมีผู้สอน)

Supervised Learning เป็นวิธีการเรียนรู้ที่โมเดลจะเรียนรู้จากชุดข้อมูลที่มีการระบุผลลัพธ์หรือเป้าหมายที่ถูกต้องไว้แล้ว (Labeled Data) โดยชุดข้อมูลจะประกอบด้วยตัวแปรต้นหรือคุณลักษณะ (Features, แทนด้วย X) และตัวแปรตามหรือผลลัพธ์ (Labels/Targets, แทนด้วย y) เป้าหมายหลักของ Supervised Learning คือการสร้างฟังก์ชัน $f(X) \approx y$ เพื่อใช้ทำนายผลลัพธ์ของข้อมูลใหม่ที่ไม่เคยพบมาก่อน (Unseen Data)

Supervised Learning สามารถแบ่งย่อยออกเป็น 2 ประเภทหลักตามลักษณะของผลลัพธ์ (y) ดังนี้

1. Classification (การจำแนกประเภท) เป็นการทำนายผลลัพธ์ที่อยู่ในรูปแบบของกลุ่มหรือหมวดหมู่ (Discrete หรือ Categorical Data) เช่น การแบ่งเป็น 2 กลุ่ม (Binary Classification) หรือมากกว่า 2 กลุ่ม (Multiclass Classification)

- **ตัวอย่างในบริบทประเทศไทย:** ธนาคารพาณิชย์ในประเทศไทยใช้ Classification ในการประเมินความเสี่ยงในการปล่อยสินเชื่อ (Credit

Scoring) โดยพิจารณาจาก Features เช่น รายได้ ประวัติการชำระหนี้ และอาชีพ เพื่อทำนาย Label ว่าลูกค้ารายนี้จะ “ชำระหนี้ตามกำหนด” หรือ “ผิดนัดชำระหนี้” (Default) หรือระบบของมหาวิทยาลัยขอนแก่นที่ใช้ทำนายความเสี่ยงที่นักศึกษาจะพ้นสภาพ (Dropout Prediction) โดยอิงจากเกรดเฉลี่ยและประวัติการเข้าเรียน

- **อัลกอริทึมที่นิยม:** Logistic Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), K-Nearest Neighbors (KNN)

```python
# ตัวอย่างการใช้ Decision Tree สำหรับปัญหา Classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd

# 1. สร้างชุดข้อมูลจำลองการพิจารณาสินเชื่อของธนาคาร
# Features: [รายได้ต่อเดือน (หมื่นบาท), จำนวนหนี้สินที่มีอยู่ (หมื่นบาท), อายุการทำงาน (ปี)]
# Label: 0 = ผิดนัดชำระหนี้, 1 = ชำระหนี้ตามกำหนด
data = {
    'Income': [3.5, 5.0, 2.0, 8.0, 4.5, 1.5, 6.0, 9.0],
    'Debt': [1.0, 2.0, 3.0, 1.5, 2.5, 2.0, 1.0, 0.5],
    'Work_Years': [2, 5, 1, 10, 4, 0.5, 7, 12],
    'Label': [1, 1, 0, 1, 0, 0, 1, 1]
}
df = pd.DataFrame(data)

X = df[['Income', 'Debt', 'Work_Years']]
y = df['Label']

# 2. แบ่งข้อมูลสำหรับฝึกสอน (Train) และทดสอบ (Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# 3. สร้างและฝึกสอนโมเดล Decision Tree
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# 4. ทำนายผลและประเมินความแม่นยำ
predictions = model.predict(X_test)
print(f"ผลการทำนาย: {predictions}")
print(f"ความแม่นยำของโมเดล: {accuracy_score(y_test, predictions):.2f}")
```

ผลลัพธ์:

```
ผลการทำนาย: [1 1]
ความแม่นยำของโมเดล: 1.00
```

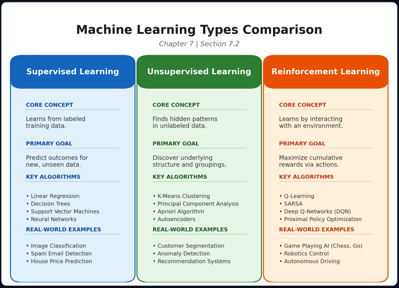
2. Regression (การวิเคราะห์การถดถอย) เป็นการทำนายผลลัพธ์ที่อยู่ในรูปแบบของตัวเลขต่อเนื่อง (Continuous Data) เป้าหมายคือการหาความสัมพันธ์ระหว่าง Features และ Label เพื่อพยากรณ์ค่าหรือปริมาณ

- **ตัวอย่างในบริบทประเทศไทย:** แพลตฟอร์มซื้อขายอสังหาริมทรัพย์ใช้ Regression ในการประเมินราคาคอนโดมิเนียมในจังหวัดขอนแก่น โดยใช้ Features เช่น ขนาดพื้นที่ (ตารางเมตร) อายุของอาคาร (ปี) และระยะห่างจากมหาวิทยาลัยขอนแก่น (กิโลเมตร) เพื่อทำนาย Label ซึ่งก็คือ “ราคาขาย” (บาท)
- **อัลกอริทึมที่นิยม:** Linear Regression, Ridge Regression, Lasso Regression, Support Vector Regression (SVR)

```python
# ตัวอย่างการใช้ Linear Regression สำหรับปัญหา Regression
from sklearn.linear_model import LinearRegression
import numpy as np

# 1. สร้างชุดข้อมูลจำลองราคาคอนโดมิเนียม
# Feature (X): ระยะห่างจากมหาวิทยาลัย (กิโลเมตร)
# Target (y): ราคาคอนโด (ล้านบาท)
X = np.array([[1.0], [2.5], [3.0], [5.0], [7.5], [10.0]])
y = np.array([3.5, 2.8, 2.7, 2.0, 1.5, 1.2]) # ยิ่งไกล ราคายิ่งลดลง

# 2. สร้างและฝึกสอนโมเดล
reg_model = LinearRegression()
reg_model.fit(X, y)

# 3. ทำนายราคาคอนโดที่อยู่ห่างออกไป 4.0 กิโลเมตร
distance_new = np.array([[4.0]])
predicted_price = reg_model.predict(distance_new)

print(f"สมการถดถอย: y = {reg_model.coef_[0]:.2f}x + {reg_model.intercept_:.2f}")
print(f"ราคาคอนโดคาดการณ์ที่ระยะห่าง 4.0 กม.: {predicted_price[0]:.2f} ล้านบาท")
```

ผลลัพธ์:

```
สมการถดถอย: y = -0.26x + 3.69
ราคาคอนโดคาดการณ์ที่ระยะห่าง 4.0 กม.: 2.67 ล้านบาท
```



Machine Learning Types Comparison

Chapter 7 | Section 7.2

	Supervised Learning	Unsupervised Learning	Reinforcement Learning
CORE CONCEPT	Learns from labeled training data.	Finds hidden patterns in unlabeled data.	Learns by interacting with an environment.
PRIMARY GOAL	Predict outcomes for new, unseen data.	Discover underlying structure and groupings.	Maximize cumulative rewards via actions.
KEY ALGORITHMS	• Linear Regression • Decision Trees • Support Vector Machines • Neural Networks	• K-Means Clustering • Principal Component Analysis • Apriori Algorithm • Autoencoders	• Q-Learning • SARSA • Deep Q-Networks (DQN) • Proximal Policy Optimization
REAL-WORLD EXAMPLES	• Image Classification • Spam Email Detection • House Price Prediction	• Customer Segmentation • Anomaly Detection • Recommendation Systems	• Game Playing AI (Chess, Go) • Robotics Control • Autonomous Driving

ประเภท Machine Learning

Unsupervised Learning (การเรียนรู้แบบไม่มีผู้สอน)

Unsupervised Learning เป็นวิธีการเรียนรู้จากชุดข้อมูลที่ไม่มีการระบุผลลัพธ์ (Unlabeled Data) ข้อมูลมีเพียงตัวแปรต้น (*X*) โดยไม่มีตัวแปรตาม (*y*) เป้าหมายหลักคือการให้โมเดลค้นหาโครงสร้างที่ซ่อนอยู่ (Hidden Structures) รูปแบบ (Patterns) หรือความสัมพันธ์ภายในข้อมูลด้วยตนเอง (VanderPlas, 2016)

Unsupervised Learning แบ่งออกเป็น 2 ประเภทหลักที่สำคัญ ดังนี้

1. Clustering (การจัดกลุ่ม) เป็นกระบวนการจัดกลุ่มข้อมูลที่มีลักษณะคล้ายคลึงกันให้อยู่ในกลุ่ม (Cluster) เดียวกัน และแยกข้อมูลที่แตกต่างกันให้อยู่คนละกลุ่ม โดยที่ระบบไม่ทราบล่วงหน้าว่ามีกลุ่มอะไรบ้าง

- **ตัวอย่างในบริบทประเทศไทย:** ธุรกิจ E-commerce หรือห้างสรรพสินค้าชั้นนำในไทย ใช้ Clustering เพื่อทำ Customer Segmentation โดยวิเคราะห์พฤติกรรมการซื้อสินค้า (เช่น ยอดใช้จ่ายรวม ความถี่ในการซื้อ) เพื่อจัดกลุ่มลูกค้า เช่น “กลุ่มลูกค้า VIP”, “กลุ่มลูกค้าที่เน้นของลดราคา” หรือ “กลุ่มลูกค้าใหม่” จากนั้นจึงนำผลลัพธ์ไปออกแบบแคมเปญการตลาดที่เหมาะสมกับแต่ละกลุ่ม
- **อัลกอริทึมที่นิยม:** K-Means, Hierarchical Clustering, DBSCAN

```
# ตัวอย่างการใช้ K-Means สำหรับปัญหา Clustering
from sklearn.cluster import KMeans
import numpy as np

# 1. สร้างชุดข้อมูลลูกค้าย้อนหลัง (ไม่มี Label)
# Features: [ความถี่ในการซื้อสินค้า (ครั้ง/เดือน), ยอดใช้จ่ายเฉลี่ย (พันบาท)]
X_customers = np.array([
    [10, 50], [12, 45], [11, 55], # กลุ่มที่ 1: ซื้อบ่อย ยอดสูง (VIP)
    [2, 5], [3, 4], [1, 6],       # กลุ่มที่ 2: นานๆ ซื้อ ยอดต่ำ
    [15, 8], [14, 7], [16, 9]     # กลุ่มที่ 3: ซื้อบ่อยแต่ยอดต่ำ
])

# 2. สร้างและฝึกสอนโมเดล K-Means โดยกำหนดให้แบ่งเป็น 3 กลุ่ม (k=3)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_customers)

# 3. ดูผลลัพธ์การจัดกลุ่ม
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

print(f"ผลการจัดกลุ่มลูกค้าแต่ละราย: {labels}")
print("จุดศูนย์กลางของแต่ละกลุ่ม (Centroids):")
for i, c in enumerate(centroids):
    print(f"กลุ่มที่ {i}: ความถี่ {c[0]:.1f} ครั้ง, ยอดใช้จ่าย {c[1]:.1f} พันบาท")
```

ผลลัพธ์:

```
ผลการจัดกลุ่มลูกค้าแต่ละราย: [2 2 2 0 0 0 1 1 1]
จุดศูนย์กลางของแต่ละกลุ่ม (Centroids):
กลุ่มที่ 0: ความถี่ 11.0 ครั้ง, ยอดใช้จ่าย 50.0 พันบาท
กลุ่มที่ 1: ความถี่ 15.0 ครั้ง, ยอดใช้จ่าย 8.0 พันบาท
กลุ่มที่ 2: ความถี่ 2.0 ครั้ง, ยอดใช้จ่าย 5.0 พันบาท
```

2. Dimensionality Reduction (การลดมิติข้อมูล) ในการวิเคราะห์ข้อมูลจริง ข้อมูลมักมีมิติ (จำนวน Features) ที่สูงมาก ซึ่งอาจนำไปสู่ปัญหา “Curse of Dimensionality” ที่ทำให้การประมวลผลช้าและโมเดลมีประสิทธิภาพลดลง การลดมิติข้อมูลคือการลดจำนวนตัวแปรลงโดยยังคงรักษาข้อมูลและโครงสร้างที่สำคัญไว้ให้ได้มากที่สุด

- **ตัวอย่างในบริบทประเทศไทย:** ในการวิเคราะห์ข้อมูลแบบสอบถามความพึงพอใจของนักศึกษามหาวิทยาลัยที่มีมากกว่า 50 คำถาม (50 มิติ) นักวิทยาศาสตร์ข้อมูลสามารถใช้ Dimensionality Reduction เพื่อรวบยอดคำถามเหล่านั้นให้เหลือเพียง 2-3 มิติหลัก (Principal Components) เช่น “ความพึงพอใจด้านวิชาการ” และ “ความพึงพอใจด้านสิ่งอำนวยความสะดวก” เพื่อนำไปแสดงผลเป็นกราฟ 2 มิติให้ผู้บริหารเข้าใจได้ง่ายขึ้น
- **อัลกอริทึมที่นิยม:** Principal Component Analysis (PCA), t-SNE, Autoencoders

Semi-supervised Learning (การเรียนรู้แบบกึ่งมีผู้สอน)

Semi-supervised Learning เป็นแนวทางที่ผสมผสานระหว่าง Supervised และ Unsupervised Learning โดยใช้ชุดข้อมูลที่มีการระบุผลลัพธ์ (Labeled Data) จำนวนน้อย ร่วมกับข้อมูลที่ไม่มีการระบุผลลัพธ์ (Unlabeled Data) จำนวนมากในการฝึกสอนโมเดล

สาเหตุที่ต้องใช้วิธีนี้เนื่องจากในโลกความเป็นจริง การได้มาซึ่ง Labeled Data มักมีต้นทุนสูง ต้องใช้เวลา และต้องอาศัยผู้เชี่ยวชาญ ในขณะที่ Unlabeled Data สามารถเก็บรวบรวมได้ง่ายและมีราคาถูกกว่ามาก

- **ตัวอย่างในบริบทประเทศไทย:** ในระบบสาธารณสุขของไทย การพัฒนาโมเดล AI เพื่อตรวจจับความผิดปกติจากภาพถ่ายเอกซเรย์ทรวงอก (Chest X-ray) เพื่อคัดกรองวัณโรค การให้รังสีแพทย์มานั่งระบุตำแหน่งความผิดปกติ (Labeling) ในภาพนับแสนภาพต้องใช้เวลาและงบประมาณมหาศาล โรงพยาบาลจึงอาจให้แพทย์ระบุผลลัพธ์เพียง 1,000 ภาพ (Labeled Data) และใช้ภาพที่เหลืออีก 99,000 ภาพ (Unlabeled Data) มาช่วยให้โมเดลเรียนรู้โครงสร้างทั่วไปของปอดผ่านกระบวนการ Semi-supervised Learning ซึ่งช่วยเพิ่มความแม่นยำของโมเดลได้อย่างมีนัยสำคัญโดยไม่ต้องใช้ต้นทุนสูง (McKinney, 2022)
- **อัลกอริทึมที่นิยม:** Self-Training, Co-Training, Graph-based methods

Reinforcement Learning (การเรียนรู้แบบเสริมกำลัง)

Reinforcement Learning (RL) เป็นกระบวนทัศน์การเรียนรู้ที่แตกต่างออกไปอย่างสิ้นเชิง โดยโมเดล (ในที่นี้เรียกว่า Agent) จะเรียนรู้วิธีการตัดสินใจหรือกระทำ (Action) ภายในสภาพแวดล้อม (Environment) เพื่อให้ได้มาซึ่งผลตอบแทนสะสมสูงสุด (Maximum Reward)

การเรียนรู้แบบนี้ไม่มีข้อมูลที่เป็นคำตอบที่ถูกต้องให้ Agent แต่ Agent ต้องเรียนรู้ผ่านการลองผิดลองถูก (Trial and Error) หากทำสิ่งที่ดีจะได้รางวัล (Reward) หากทำสิ่งที่ไม่ดีจะถูกลงโทษ (Penalty) คล้ายกับการฝึกสัตว์เลี้ยง

- **ตัวอย่างในบริบทประเทศไทย:**
  - **การจัดการจราจร:** การนำ AI มาควบคุมสัญญาณไฟจราจรอัจฉริยะบริเวณสี่แยกมะลิวัลย์ จังหวัดขอนแก่น Agent คือระบบควบคุมไฟจราจร Action คือการเปลี่ยนสีไฟ Reward คือจำนวนรถที่ระบายได้และ Penalty คือระยะเวลาที่รถสะสมในแต่ละฝั่ง ระบบจะเรียนรู้ปรับเปลี่ยนเวลาไฟเขียวไฟแดงให้เหมาะสมกับปริมาณรถแบบ Real-time เพื่อลดปัญหาการจราจรติดขัด
  - **การเงิน:** บริษัทหลักทรัพย์ในไทยใช้ RL ในการสร้าง Algorithmic Trading Bot โดยให้ Bot เรียนรู้ที่จะซื้อหรือขายหุ้น (Action) ในสภาวะตลาดต่างๆ (Environment) เพื่อสร้างผลกำไรสูงสุด (Reward)
- **อัลกอริทึมที่นิยม:** Q-Learning, Deep Q-Network (DQN), Proximal Policy Optimization (PPO)

ตารางสรุปประเภทของ Machine Learning

เพื่อความเข้าใจที่ชัดเจนยิ่งขึ้น ตารางด้านล่างแสดงการเปรียบเทียบประเภทของ Machine Learning ในมิติต่างๆ

ประเภทของ ML	ลักษณะของ Input	ลักษณะของ Output	ตัวอย่างการประยุกต์ใช้	อัลกอริทึมที่เกี่ยวข้อง

Supervised Learning (Classification)	ข้อมูลมี Label (*X*, *y*)	หมวดหมู่ / กลุ่ม (Discrete)	กรองอีเมลขยะ, ทำนายการผิดนัดชำระหนี้, วินิจฉัยโรคจากอาการ	Logistic Regression, Decision Tree, SVM
Supervised Learning (Regression)	ข้อมูลมี Label (*X*, *y*)	ตัวเลขต่อเนื่อง (Continuous)	พยากรณ์ราคาบ้าน, พยากรณ์ยอดขายล่วงหน้า, ประเมินอุณหภูมิ	Linear Regression, Ridge, Lasso
Unsupervised Learning (Clustering)	ข้อมูลไม่มี Label (*X* เท่านั้น)	โครงสร้างการจัดกลุ่ม	แบ่งกลุ่มลูกค้า (Segmentation), จัดกลุ่มบทความข่าวที่คล้ายกัน	K-Means, DBSCAN, Hierarchical
Unsupervised Learning (Dim. Reduction)	ข้อมูลไม่มี Label (*X* เท่านั้น)	ข้อมูลที่มีมิติลดลง	บีบอัดข้อมูล, การแสดงผลข้อมูลมิติสูงให้เป็นกราฟ 2D/3D	PCA, t-SNE, SVD
Semi-supervised Learning	มี Label จำนวนน้อยและไม่มี Label จำนวนมาก	คล้าย Supervised (ทำนาย Label)	วิเคราะห์ภาพถ่ายทางการแพทย์ขนาดใหญ่, จัดหมวดหมู่เว็บไซต์	Self-Training, Co-Training
Reinforcement Learning	สภาพแวดล้อม (State)	การกระทำ (Action) ที่ได้ Reward สูงสุด	บอทเล่นเกม, ควบคุมไฟจราจรอัจฉริยะ, หุ่นยนต์เดินอัตโนมัติ	Q-Learning, DQN, PPO

สรุปประเด็นสำคัญ

- **Supervised Learning** เรียนรู้จากข้อมูลที่มีเฉลย (Label) เพื่อสร้างโมเดลทำนายผลลัพธ์ แบ่งเป็น Classification (ทำนายหมวดหมู่) และ Regression (ทำนายตัวเลข)
- **Unsupervised Learning** เรียนรู้จากข้อมูลที่ไม่มีเฉลย เพื่อค้นหาโครงสร้างหรือรูปแบบที่ซ่อนอยู่ เช่น การจัดกลุ่ม (Clustering) และการลดมิติข้อมูล (Dimensionality Reduction)
- **Semi-supervised Learning** ใช้ข้อมูลที่มีเฉลยจำนวนน้อยร่วมกับข้อมูลที่ไม่มีเฉลยจำนวนมาก เพื่อประหยัดต้นทุนในการทำ Label แต่ยังคงประสิทธิภาพในการทำนาย
- **Reinforcement Learning** เป็นการเรียนรู้ผ่านการลองผิดลองถูกของ Agent ในสภาพแวดล้อม โดยมุ่งเน้นที่การตัดสินใจเพื่อให้ได้รับผลตอบแทน (Reward) สูงสุด

คำถามท้าย Section

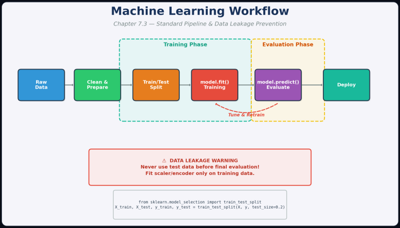
1. หากนักศึกษาได้รับมอบหมายจากโรงพยาบาลศรีนครินทร์ ให้สร้างระบบ AI เพื่อทำนายว่าผู้ป่วยรายใหม่มีโอกาสเป็นโรคเบาหวานหรือไม่ (เป็น/ไม่เป็น) โดยอาศัยข้อมูลประวัติสุขภาพในอดีตของผู้ป่วยนับหมื่นรายที่มีการบันทึกผลการวินิจฉัยไว้แล้ว นักศึกษาจะเลือกใช้ Machine Learning ประเภทใด และอยู่ในกลุ่มย่อยใด จงอธิบายเหตุผล
2. พิจารณาสถานการณ์ที่บริษัท E-commerce รายใหญ่ในไทยต้องการลดค่าใช้จ่ายในการจัดเก็บข้อมูล (Storage) และเร่งความเร็วในการประมวลผลข้อมูลพฤติกรรมผู้บริโภคที่มีมากกว่า 1,000 ตัวแปร (Features) โดยไม่ต้องการสูญเสียสาระสำคัญของข้อมูล นักศึกษาจะใช้วิธีการใดในกลุ่ม Unsupervised Learning เพื่อแก้ปัญหานี้
3. จงเปรียบเทียบความแตกต่างระหว่างวิธีคิดของ Supervised Learning และ Reinforcement Learning ในบริบทของการสร้างโปรแกรมเล่นหมากรุก (Chess AI)

ML Workflow

การพัฒนาระบบการเรียนรู้ของเครื่อง (Machine Learning) ให้มีประสิทธิภาพและสามารถนำไปใช้งานได้จริงนั้น จำเป็นต้องอาศัยกระบวนการทำงานที่เป็นระบบและมีมาตรฐาน กระบวนการดังกล่าวเรียกว่า ML Workflow ซึ่งเป็นลำดับขั้นตอนตั้งแต่การนำข้อมูลเข้าสู่ระบบ ไปจนถึงการประเมินประสิทธิภาพของโมเดล (VanderPlas, 2016) การทำความเข้าใจ Workflow นี้อย่างถ่องแท้เป็นสิ่งจำเป็นสำหรับนักวิทยาการคอมพิวเตอร์ เนื่องจากจะช่วยป้องกันข้อผิดพลาดเชิงตรรกะที่อาจเกิดขึ้นระหว่างการพัฒนาโมเดล

พิจารณาแผนภาพด้านล่าง ซึ่งแสดงกระบวนการทำงานมาตรฐานของ Machine Learning (Standard ML Workflow)

```
+-----------------+      +-----------------+      +-----------------+

|                 |      |                 |      |                 |
|   1. Raw Data   | ---> |   2. Split Data | ---> | Training Data   |
|                 |      |   (Train/Test)  |      | Testing Data    |
+-----------------+      +-----------------+      +-----------------+
                                                           |
                                                           v
+-----------------+      +-----------------+      +-----------------+

|                 |      |                 |      |                 |
|  5. Evaluate    | <--- |   4. Predict    | <--- |   3. Fit (Train)|
|  (Score/Metrics)|      |   (Inference)   |      |   Model         |
+-----------------+      +-----------------+      +-----------------+
```



Machine Learning Workflow

Chapter 7.3 — Standard Pipeline & Data Leakage Prevention

⚠ DATA LEAKAGE WARNING
Never use test data before final evaluation!
Fit scaler/encoder only on training data.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

ML Workflow และ Data Leakage Prevention

การแบ่งข้อมูล (Train-Test Split) และความสามารถในการวางนัยทั่วไป (Generalization)

เป้าหมายสูงสุดของการสร้างโมเดล Machine Learning ไม่ใช่การสร้างโมเดลที่จดจำข้อมูลในอดีตได้สมบูรณ์แบบ แต่คือการสร้างโมเดลที่มี **ความสามารถในการวางนัยทั่วไป (Generalization)** ซึ่งหมายถึงความสามารถของโมเดลในการทำนายผลลัพธ์ได้อย่างแม่นยำเมื่อเผชิญกับ "ข้อมูลใหม่ที่ไม่เคยเห็นมาก่อน" (Unseen Data) (James et al., 2023)

เพื่อวัดความสามารถในการวางนัยทั่วไป เราจึงไม่สามารถประเมินโมเดลด้วยข้อมูลชุดเดียวกันกับที่ใช้สอนโมเดลได้ ด้วยเหตุนี้ ขั้นตอนแรกที่สำคัญที่สุดคือการแบ่งข้อมูล (Data Splitting) ออกเป็นสองส่วนหลัก ได้แก่:

ประเภทข้อมูล	สัดส่วนทั่วไป	วัตถุประสงค์การใช้งาน	เปรียบเทียบกับการเรียนของนักศึกษา
Training Data	70% - 80%	ใช้สำหรับสอนโมเดล (Fit) ให้เรียนรู้รูปแบบและปรับค่าพารามิเตอร์	แบบฝึกหัดที่อาจารย์ให้ทำในห้องเรียนพร้อมเฉลย
Testing Data	20% - 30%	ใช้สำหรับทดสอบโมเดล (Evaluate) ห้ามนำไปให้โมเดลเรียนรู้โดยเด็ดขาด	ข้อสอบปลายภาคที่นักศึกษาไม่เคยเห็นมาก่อน

ตัวอย่างในบริบทมหาวิทยาลัย: พิจารณาการสร้างโมเดลทำนายโอกาสการพ้นสภาพนักศึกษา (Dropout Prediction) ของคณะวิทยาการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น หากเรานำข้อมูลนักศึกษาชั้นปีที่ 1-4 ทั้งหมดไปสอนโมเดล และทดสอบด้วยข้อมูลชุดเดิม โมเดลอาจจดจำได้ว่า "รหัสนักศึกษา 653040xxx-x พ้นสภาพ" โดยไม่ได้เรียนรู้ถึงสาเหตุที่แท้จริง (เช่น เกรดเฉลี่ยต่ำ ขาดเรียนบ่อย) เมื่อนำโมเดลนี้ไปใช้กับนักศึกษารหัส 66 ที่เพิ่งเข้ามาใหม่ โมเดลจะทำนายผิดพลาดอย่างสิ้นเชิง นี่คือภาวะที่เรียกว่า Overfitting ซึ่งเกิดจากการขาด Generalization

ความสามารถในการทำซ้ำ (Reproducibility) และ `random_state`

ในการสุ่มแบ่งข้อมูลด้วยคอมพิวเตอร์ อัลกอริทึมจะใช้ตัวสร้างเลขสุ่มเทียม (Pseudo-Random Number Generator) หากไม่มีการควบคุมการรันโค้ดแต่ละครั้งจะได้ชุดข้อมูล Training และ Testing ที่แตกต่างกัน ส่งผลให้ประสิทธิภาพของโมเดลเปลี่ยนไปมา ทำให้ยากต่อการเปรียบเทียบและพัฒนาโมเดล

เพื่อรักษา **ความสามารถในการทำซ้ำ (Reproducibility)** ซึ่งเป็นหัวใจสำคัญของกระบวนการทางวิทยาศาสตร์ เราจำเป็นต้องกำหนดค่า Seed ให้กับตัวสร้างเลขสุ่ม ซึ่งในไลบรารี Scikit-Learn จะใช้พารามิเตอร์ที่เรียกว่า `random_state` การกำหนดค่านี้ (เช่น `random_state=42`) จะรับประกันว่า ไม่ว่านักศึกษาจะรันโค้ดกี่ครั้ง หรือนำโค้ดไปรันบนเครื่องคอมพิวเตอร์เครื่องอื่น การแบ่งข้อมูลก็จะได้ผลลัพธ์เหมือนเดิมทุกประการ (McKinney, 2022)

การสอนโมเดล (Fit) และ การทำนาย (Predict)

เมื่อแบ่งข้อมูลเรียบร้อยแล้ว กระบวนการถัดไปคือการนำข้อมูลเข้าสู่โมเดล ซึ่งใน Scikit-Learn API จะมีโครงสร้างการทำงานที่ชัดเจนและเป็นมาตรฐานเดียวกันดังนี้ (VanderPlas, 2016):

1. **การสอนโมเดล (Fit):** ใช้คำสั่ง `model.fit(X_train, y_train)` ในขั้นตอนนี้ โมเดลจะนำฟีเจอร์ (Features หรือ `X_train`) และผลเฉลย (Labels หรือ `y_train`) ไปคำนวณทางคณิตศาสตร์ เพื่อค้นหาความสัมพันธ์และปรับน้ำหนัก (Weights) ภายในโมเดล ขั้นตอนนี้เปรียบเสมือนการอ่านหนังสือเตรียมสอบ
2. **การทำนาย (Predict):** ใช้คำสั่ง `model.predict(X_test)` เมื่อโมเดลเรียนรู้เสร็จแล้ว เราจะป้อนเฉพาะฟีเจอร์ของข้อมูลชุดทดสอบ (`X_test`) ให้กับโมเดล โดยไม่ให้เห็นผลเฉลย โมเดลจะใช้ความรู้ที่ได้จากขั้นตอน Fit มาสร้างผลลัพธ์การทำนาย (`y_pred`) ขั้นตอนนี้เปรียบเสมือนการทำข้อสอบ

การประเมินผล (Evaluate)

ขั้นตอนสุดท้ายคือการนำผลการทำนาย (`y_pred`) ที่ได้จากโมเดล มาเปรียบเทียบกับผลเฉลยจริง (`y_test`) ที่เราเก็บซ่อนไว้ เพื่อคำนวณหาประสิทธิภาพของโมเดล ตัวชี้วัด (Metrics) ที่ใช้จะแตกต่างกันไปตามประเภทของปัญหา เช่น ปัญหาการจำแนกประเภท (Classification) อาจใช้ค่าความแม่นยำ (Accuracy) ในขณะที่ปัญหาการถดถอย (Regression) อาจใช้ค่าความคลาดเคลื่อนกำลังสองเฉลี่ย (Mean Squared Error)

ปัญหาข้อมูลรั่วไหล (Data Leakage)

หนึ่งในข้อผิดพลาดที่ร้ายแรงที่สุดและพบได้บ่อยที่สุดในหมู่นักวิทยาศาสตร์ข้อมูลมือใหม่ คือ **ปัญหาข้อมูลรั่วไหล (Data Leakage)**

Data Leakage เกิดขึ้นเมื่อข้อมูลจากชุดทดสอบ (Testing Data) หรือข้อมูลในอนาคตที่โมเดลไม่ควรจะรู้ในขณะที่ทำการทำนาย ได้รั่วไหลเข้าไปอยู่ในกระบวนการสอนโมเดล (Training Process) ผลที่ตามมาคือ โมเดลจะมีประสิทธิภาพสูงมากในขั้นตอนการทดสอบ (ทดสอบได้ความแม่นยำ 99% หรือ 100%) แต่เมื่อนำไปใช้งานจริง (Production) โมเดลกลับทำงานล้มเหลวโดยสิ้นเชิง (Grus, 2019)

ประเภทของ Data Leakage ที่พบบ่อย:

1. **Target Leakage:** การนำตัวแปรที่เป็นผลพวงจากเป้าหมาย (Target) มาใช้เป็นฟีเจอร์
   - *ตัวอย่างบริบทธนาคารไทย:* ธนาคารต้องการสร้างโมเดลทำนายว่าลูกค้าจะเบี้ยวหนี้ (Default) หรือไม่ หากนักศึกษานำฟีเจอร์ "ค่าปรับล่าช้าสะสม" (Late Fee Accrued) ใส่เข้าไปในโมเดลด้วย จะเกิด Target Leakage ทันที เพราะลูกค้าจะโดนค่าปรับล่าช้าก็ต่อเมื่อพวกเขาเริ่มเบี้ยวหนี้ไปแล้ว ในความเป็นจริง ณ วันที่ธนาคารต้องตัดสินใจอนุมัติสินเชื่อ ข้อมูลค่าปรับล่าช้ายังไม่เกิดขึ้น
2. **Train-Test Contamination:** การทำกระบวนการเตรียมข้อมูล (Data Preprocessing) เช่น การทำ Normalization หรือ การเติมค่าสูญหาย (Imputation) กับข้อมูล *ทั้งชุด* ก่อนที่จะทำการแบ่ง Train-Test Split
   - *ตัวอย่าง:* หากนักศึกษาหาค่าเฉลี่ยของรายได้ลูกค้าจากข้อมูลทั้งชุด (รวมข้อมูล Test ด้วย) แล้วนำไปเติมในช่องที่ว่างเปล่า เท่ากับว่าข้อมูลสถิติของชุด Test ได้รั่วไหลเข้าไปผสมในชุด Train แล้ว วิธีที่ถูกต้องคือ ต้องแบ่ง Train-Test ก่อน จากนั้นหาค่าเฉลี่ย *เฉพาะในชุด Train* แล้วนำค่านั้นไปเติมทั้งในชุด Train และ ชุด Test

การนำไปปฏิบัติด้วย Python (Implementation in Python)

เพื่อทำความเข้าใจ ML Workflow อย่างเป็นรูปธรรม พิจารณาตัวอย่างการเขียนโปรแกรมด้วยภาษา Python และไลบรารี Scikit-Learn โดยจำลองสถานการณ์ของบริษัท E-commerce ในประเทศไทย ที่ต้องการทำนายว่าลูกค้าจะเลิกใช้งานแพลตฟอร์มหรือไม่ (Customer Churn Prediction)

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. การเตรียมข้อมูล (Raw Data)
# จำลองข้อมูลลูกค้า E-commerce (อายุ, ความถี่ในการซื้อต่อเดือน, จำนวนครั้งที่ติดต่อ Call Center, เลิกใช้งานหรือไม่)
data = {
    'Age': [25, 34, 45, 23, 50, 31, 29, 40, 36, 22],
    'Purchase_Frequency': [15, 2, 20, 1, 25, 3, 18, 2, 12, 1],
    'Support_Calls': [0, 5, 1, 4, 0, 6, 1, 4, 2, 4],
    'Churn': [0, 1, 0, 1, 0, 1, 0, 1, 0, 1] # 0 = ใช้งานต่อ, 1 = เลิกใช้งาน
}
df = pd.DataFrame(data)

# แยก Features (X) และ Target (y)
X = df.drop('Churn', axis=1)
y = df['Churn']

# 2. การแบ่งข้อมูล (Split Data)
# แบ่งข้อมูลเป็น Train 70% และ Test 30% พร้อมกำหนด random_state เพื่อ Reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42 # กำหนด Seed ให้คงที่
)
```

```
print(f"จำนวนข้อมูล Training: {len(X_train)} แถว")
print(f"จำนวนข้อมูล Testing: {len(X_test)} แถว")

# 3. การสอนโมเดล (Fit Model)
# เลือกใช้อัลกอริทึม Random Forest
model = RandomForestClassifier(random_state=42)

# ให้โมเดลเรียนรู้จากข้อมูล Training เท่านั้น
model.fit(X_train, y_train)

# 4. การทำนาย (Predict)
# นำข้อมูล Testing (ที่โมเดลไม่เคยเห็น) ไปทำนายผล
y_pred = model.predict(X_test)

# 5. การประเมินผล (Evaluate)
# เปรียบเทียบผลการทำนาย (y_pred) กับผลเฉลยจริง (y_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"ผลการทำนาย (Predictions): {y_pred}")
print(f"ผลเฉลยจริง (Actuals):      {y_test.values}")
print(f"ความแม่นยำของโมเดล (Accuracy): {accuracy * 100:.2f}%")
```

ผลลัพธ์:

```
จำนวนข้อมูล Training: 7 แถว
จำนวนข้อมูล Testing: 3 แถว

ผลการทำนาย (Predictions): [0 1 1]
ผลเฉลยจริง (Actuals):      [1 0 0]
ความแม่นยำของโมเดล (Accuracy): 0.00%
```

> **หมายเหตุ:** ค่า Accuracy ที่ได้อาจต่ำมากในตัวอย่างนี้ เนื่องจากชุดข้อมูลมีขนาดเล็กมาก (เพียง 10 แถว) ซึ่งไม่เพียงพอต่อการเรียนรู้รูปแบบที่แท้จริงของข้อมูล ในการใช้งานจริง ชุดข้อมูลควรมีขนาดใหญ่กว่านี้อย่างมาก

จากการศึกษาโค้ดด้านบน จะเห็นได้ว่ากระบวนการทำงานถูกแบ่งออกเป็น 5 ขั้นตอนอย่างชัดเจน การปฏิบัติตาม Workflow นี้อย่างเคร่งครัด โดยเฉพาะการทำ `train_test_split` ก่อนที่จะดำเนินการใดๆ กับข้อมูล จะช่วยให้โมเดลที่นักศึกษาพัฒนาขึ้นมีความน่าเชื่อถือ และสามารถนำไปประยุกต์ใช้ในโครงงานหรือการทำงานจริงได้อย่างมีประสิทธิภาพ

สรุปประเด็นสำคัญ

- **Standard ML Workflow** ประกอบด้วย 5 ขั้นตอนหลัก: การนำเข้าข้อมูล (Data) -> การแบ่งข้อมูล (Split) -> การสอนโมเดล (Fit) -> การทำนาย (Predict) -> การประเมินผล (Evaluate)
- **Generalization** คือเป้าหมายหลักของการสร้างโมเดล ซึ่งวัดจากความสามารถในการทำนายข้อมูลใหม่ที่โมเดลไม่เคยเห็นมาก่อน (Unseen Data)
- **Reproducibility** เป็นสิ่งสำคัญในการทดลองทางวิทยาการคอมพิวเตอร์ การกำหนดพารามิเตอร์ `random_state` ช่วยให้การสุ่มแบ่งข้อมูลได้ผลลัพธ์เหมือนเดิมทุกครั้ง
- **Data Leakage** คือข้อผิดพลาดร้ายแรงที่ข้อมูลในอนาคตหรือข้อมูลจากชุดทดสอบ รั่วไหลเข้าไปในกระบวนการสอนโมเดล ทำให้ผลการประเมินดีเกินจริงแต่ใช้งานจริงไม่ได้

คำถามท้าย Section

1. หากนักศึกษาทำการปรับค่ามาตรฐานของข้อมูล (Standardization) โดยใช้ฟังก์ชัน `StandardScaler().fit_transform(X)` กับข้อมูลทั้งหมดก่อนที่จะเข้าสู่ฟังก์ชัน `train_test_split()` การกระทำนี้จะส่งผลเสียต่อโมเดลอย่างไร และเกี่ยวข้องกับแนวคิดใดในบทเรียน?
2. พิจารณาสถานการณ์: โรงพยาบาลศรีนครินทร์ต้องการสร้างโมเดลทำนายการติดเชื้อในกระแสเลือดของผู้ป่วย โดยใช้ข้อมูลประวัติการรักษา หากในตารางข้อมูลมีคอลัมน์ “รหัสยาปฏิชีวนะที่สั่งจ่ายเพื่อรักษาการติดเชื้อ” นักศึกษาควรนำคอลัมน์นี้มาเป็น Feature ในการสอนโมเดลหรือไม่? จงอธิบายเหตุผลโดยใช้แนวคิดเรื่อง Data Leakage

---

Bias-Variance Tradeoff

ในการสร้างโมเดล Machine Learning เป้าหมายสูงสุดคือการสร้างโมเดลที่มีความสามารถในการทำนายข้อมูลใหม่ที่โมเดลไม่เคยเห็นมาก่อนได้อย่างแม่นยำ (Generalization) อย่างไรก็ตาม ในกระบวนการเรียนรู้ของโมเดลจากชุดข้อมูลฝึกสอน (Training Data) มักจะเผชิญกับปัญหาพื้นฐานที่สำคัญที่สุดประการหนึ่งในสาขาวิทยาการข้อมูล ซึ่งรู้จักกันในชื่อ **Bias-Variance Tradeoff** (James et al., 2023)

เพื่อทำความเข้าใจปัญหานี้ จำเป็นต้องพิจารณาองค์ประกอบของความคลาดเคลื่อนในการทำนาย (Prediction Error) ซึ่งประกอบด้วย 3 ส่วนหลัก ได้แก่:

1. **ความเอนเอียง (Bias):** ความคลาดเคลื่อนที่เกิดจากข้อสมมติฐานของโมเดลที่ง่ายเกินไปเมื่อเทียบกับความเป็นจริงของข้อมูล
2. **ความแปรปรวน (Variance):** ความคลาดเคลื่อนที่เกิดจากความไว (Sensitivity) ของโมเดลต่อความผันผวนย่อยๆ หรือสัญญาณรบกวน (Noise) ในชุดข้อมูลฝึกสอน
3. **ความคลาดเคลื่อนที่ไม่สามารถลดทอนได้ (Irreducible Error):** ความคลาดเคลื่อนที่เกิดจากปัจจัยภายนอกหรือตัวแปรที่ไม่ได้ถูกเก็บข้อมูลมา ซึ่งไม่สามารถแก้ไขได้ด้วยการปรับปรุงโมเดล

ในทางคณิตศาสตร์ ค่าคาดหมายของความคลาดเคลื่อนกำลังสองเฉลี่ย (Expected Mean Squared Error: MSE) ของโมเดลสามารถเขียนในรูปสมการได้ดังนี้:

$$Expected\ MSE = Bias^2 + Variance + Irreducible\ Error$$

เนื่องจาก Irreducible Error เป็นค่าคงที่ การจะลดความคลาดเคลื่อนรวมให้ต่ำที่สุดจึงต้องบริหารจัดการค่า Bias และ Variance ซึ่งทั้งสองค่านี้มีพฤติกรรมที่แปรผกผันกัน การทำความเข้าใจและหาจุดสมดุลระหว่างสองสิ่งนี้คือหัวใจสำคัญของการสร้างโมเดลที่มีประสิทธิภาพ

ความเอนเอียง (Bias) และภาวะ Underfitting

ความเอนเอียง (Bias) คือความแตกต่างระหว่างค่าเฉลี่ยของผลลัพธ์ที่โมเดลทำนายได้ กับค่าจริงของข้อมูล โมเดลที่มีค่า Bias สูง (High Bias) มักจะเกิดจากการเลือกใช้โมเดลที่มีความซับซ้อนน้อยเกินไป (Low Model Complexity) หรือมีข้อสมมติฐานที่เข้มงวดเกินไป ทำให้โมเดลไม่สามารถเรียนรู้รูปแบบหรือความสัมพันธ์ที่แท้จริงที่ซ่อนอยู่ในข้อมูลได้

สภาวะที่โมเดลมีค่า Bias สูงและไม่สามารถจับความสัมพันธ์ของข้อมูลได้นี้ เรียกว่า **Underfitting**

ตัวอย่างในบริบทประเทศไทย: พิจารณาการสร้างโมเดลเพื่อประเมินราคาประเมินที่ดินในเขตกรุงเทพมหานคร หากนักวิทยาศาสตร์ข้อมูลเลือกใช้โมเดล Linear Regression แบบง่าย โดยใช้เพียงตัวแปร “ระยะห่างจากสถานีรถไฟฟ้า” เพียงตัวแปรเดียวในการทำนายราคา โมเดลนี้จะมี High Bias ทันที เนื่องจากในความเป็นจริง ราคาที่ดินขึ้นอยู่กับปัจจัยที่ซับซ้อนมากมาย เช่น ขนาดพื้นที่ กฎหมายผังเมือง ความกว้างของถนนหน้าแปลงที่ดิน ฯลฯ โมเดลที่เรียบง่ายเกินไปนี้จะทำนายผิดพลาดสูงทั้งบนชุดข้อมูลฝึกสอน (Training Set) และชุดข้อมูลทดสอบ (Test Set)

ลักษณะสำคัญของโมเดลที่มีภาวะ Underfitting:

- มีค่าความคลาดเคลื่อนสูงในชุดข้อมูลฝึกสอน (High Training Error)
- มีค่าความคลาดเคลื่อนสูงในชุดข้อมูลทดสอบ (High Testing Error)
- โมเดลมีโครงสร้างที่เรียบง่ายเกินไป (เช่น การพยายามใช้เส้นตรงลากผ่านข้อมูลที่มีการกระจายตัวแบบเส้นโค้ง)

ความแปรปรวน (Variance) และภาวะ Overfitting

ความแปรปรวน (Variance) บ่งบอกถึงปริมาณความเปลี่ยนแปลงของผลการทำนาย หากเราเปลี่ยนชุดข้อมูลฝึกสอนไปเป็นชุดข้อมูลอื่น โมเดลที่มีค่า Variance สูง (High Variance) คือโมเดลที่พยายามเรียนรู้และจดจำทุกรายละเอียดในชุดข้อมูลฝึกสอน รวมถึงสัญญาณรบกวน (Noise) และค่าผิดปกติ (Outliers) ด้วย โมเดลประเภทนี้จะมีความซับซ้อนสูงมาก (High Model Complexity)

สภาวะที่โมเดลเรียนรู้ชุดข้อมูลฝึกสอนได้ดีเยี่ยม แต่ไม่สามารถนำไปใช้ทำนายข้อมูลใหม่ได้ เรียกว่า **Overfitting**

ตัวอย่างในบริบทประเทศไทย: พิจารณาระบบแนะนำสินค้า (Recommendation System) ของแพลตฟอร์ม E-commerce ในไทย หากโมเดลถูกสร้างให้มีความซับซ้อนสูงมาก โดยจดจำพฤติกรรมว่า “ผู้ใช้เพศหญิง อายุ 21 ปี ที่เข้าสู่ระบบผ่านมือถือระบบ Android ในวันพุธ เวลา 14:30 น. จากจังหวัดขอนแก่น จะต้องซื้อครีมกันแดดยี่ห้อ A” โมเดลนี้อาจทำนายข้อมูลในอดีตได้แม่นยำ 100% แต่เมื่อนำไปใช้จริงกับข้อมูลผู้ใช้คนใหม่ที่มีลักษณะคล้ายกันแต่เวลาเปลี่ยนไปเพียงเล็กน้อย โมเดลจะทำนายผิดพลาดทันที เพราะโมเดล “ท่องจำ” ข้อมูลมากเกินไปแทนที่จะเรียนรู้ “รูปแบบทั่วไป” (General Pattern)

ลักษณะสำคัญของโมเดลที่มีภาวะ Overfitting:

- มีค่าความคลาดเคลื่อนต่ำมากในชุดข้อมูลฝึกสอน (Low Training Error)
- มีค่าความคลาดเคลื่อนสูงในชุดข้อมูลทดสอบ (High Testing Error)
- โมเดลมีโครงสร้างที่ซับซ้อนเกินความจำเป็น (เช่น โครงข่ายประสาทเทียมที่มีจำนวนชั้นและโหนดมากเกินไปสำหรับชุดข้อมูลขนาดเล็ก)

การหาจุดสมดุล (The Sweet Spot) และความสามารถในการวางนัยทั่วไป (Generalization)

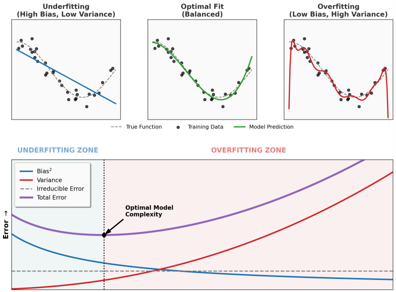
ความท้าทายหลักของ Machine Learning คือ **การแลกเปลี่ยน (Tradeoff)** ระหว่าง Bias และ Variance (VanderPlas, 2016)

- หากต้องการลด Bias ต้องเพิ่มความซับซ้อนให้โมเดล ซึ่งจะส่งผลให้ Variance เพิ่มขึ้น
- หากต้องการลด Variance ต้องลดความซับซ้อนของโมเดล ซึ่งจะส่งผลให้ Bias เพิ่มขึ้น

เป้าหมายคือการหา **จุดสมดุล (Sweet Spot)** ซึ่งเป็นระดับความซับซ้อนของโมเดลที่ทำให้ “ความคลาดเคลื่อนรวม” (Total Error) บนข้อมูลชุดทดสอบมีค่าน้อยที่สุด ณ จุดนี้ โมเดลจะมีความสามารถในการวางนัยทั่วไป (Generalization) สูงสุด หมายความว่าโมเดลเรียนรู้โครงสร้างหลักของข้อมูลได้ดีพอ (Bias ต่ำในระดับที่ยอมรับได้) และไม่จดจำสัญญาณรบกวนมากเกินไป (Variance ต่ำในระดับที่ยอมรับได้)

สามารถอธิบายความสัมพันธ์นี้ได้ผ่านแผนภาพด้านล่าง:

```
     High |                                        .
          |                                       / \
          |                                      /   \ ⟵ Total Error (Test Error)
          |                                     /     \
  Error   |                                    /       \
          |     \                            _/         \
          |      \                          /            \ ⟵ Variance
          |       \                        /              \
          |        \      Sweet Spot      /                \
          |         \        |           /                  \
          |          \___    V          /                    \
          |              \___._        /                      \
          |                  | \_     /                        \
          |                  |   \___/                          \
          |                  |       \__                         \
          |                  |          \____                     \ ⟵ Bias^2
     Low  |..................|...............\_____________________\_________
          +------------------------------------------------------------------
            Low                                                      High
                               Model Complexity
            (Underfitting)                                     (Overfitting)
```



Bias-Variance Tradeoff

ตารางที่ 7.1: สรุปความสัมพันธ์ระหว่างความซับซ้อนของโมเดลและภาวะของโมเดล

ความซับซ้อนของโมเดล (Complexity)	ระดับ Bias	ระดับ Variance	Training Error	Test Error	ภาวะที่เกิดขึ้น (State)
ต่ำเกินไป	สูง (High)	ต่ำ (Low)	สูง	สูง	Underfitting
เหมาะสม (Sweet Spot)	**ต่ำ (Low)**	**ต่ำ (Low)**	**ต่ำ**	**ต่ำ**	**Good Generalization**
สูงเกินไป	ต่ำ (Low)	สูง (High)	ต่ำมาก (ใกล้ 0)	สูง	Overfitting

การจำลองภาพปัญหา: Polynomial Regression (Degree 1, 4, 15)

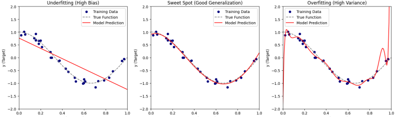
เพื่อให้เห็นภาพของ Bias-Variance Tradeoff อย่างเป็นรูปธรรม เราจะใช้เทคนิค **Polynomial Regression** ซึ่งเป็นการปรับโมเดลเชิงเส้น (Linear Model) ให้สามารถสร้างเส้นโค้งเพื่อฟิต (Fit) กับข้อมูลได้ โดยการเพิ่มระดับ (Degree) ของสมการพหุนาม

- **Degree 1:** สมการเส้นตรง ($y = ax + b$) แทนโมเดลที่มีความซับซ้อนต่ำ
- **Degree 4:** สมการกำลังสี่ ($y = ax^4 + bx^3 + cx^2 + dx + e$) แทนโมเดลที่มีความซับซ้อนปานกลาง
- **Degree 15:** สมการกำลังสิบห้า แทนโมเดลที่มีความซับซ้อนสูงมาก

ในตัวอย่างนี้ เราจะสร้างข้อมูลจำลองที่มีความสัมพันธ์แบบฟังก์ชันโคไซน์ (Cosine) โดยมีการผสมสัญญาณรบกวน (Noise) เข้าไป เพื่อจำลองข้อมูลในโลกความเป็นจริง

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# 1. สร้างชุดข้อมูลจำลอง (Simulated Data)
np.random.seed(42) # กำหนด seed เพื่อให้ผลลัพธ์เหมือนเดิมทุกครั้งที่รัน

# ฟังก์ชันที่แท้จริง (True Function) ที่เราต้องการให้โมเดลเรียนรู้
def true_function(X):
    return np.cos(1.5 * np.pi * X)

# สุ่มข้อมูล X จำนวน 30 จุด ในช่วง [0, 1]
n_samples = 30
X = np.sort(np.random.rand(n_samples))
# สร้างค่า y โดยนำฟังก์ชันจริงบวกด้วยสัญญาณรบกวน (Noise)
y = true_function(X) + np.random.randn(n_samples) * 0.1

# สร้างข้อมูล X_test สำหรับวาดเส้นกราฟให้ต่อเนื่อง
X_test = np.linspace(0, 1, 100)

# 2. กำหนดระดับของ Polynomial ที่ต้องการทดสอบ
degrees = [1, 4, 15]

plt.figure(figsize=(15, 5))

# 3. วนลูปสร้างโมเดลและพล็อตผลลัพธ์
for i, degree in enumerate(degrees):
    ax = plt.subplot(1, len(degrees), i + 1)

    # สร้าง Pipeline: แปลงข้อมูลเป็น Polynomial ก่อน แล้วจึงใช้ Linear Regression
    # การใช้ Pipeline ช่วยให้โค้ดเป็นระเบียบและลดข้อผิดพลาด (McKinney, 2022)
    polynomial_features = PolynomialFeatures(degree=degree, include_bias=False)
    linear_regression = LinearRegression()
    pipeline = make_pipeline(polynomial_features, linear_regression)

    # ฝึกสอน (Train) โมเดล
    pipeline.fit(X[:, np.newaxis], y)

    # ทำนายผลบนชุดข้อมูล X_test
    y_pred = pipeline.predict(X_test[:, np.newaxis])

    # พล็อตข้อมูลจริง (จุด scatter) และฟังก์ชันที่แท้จริง (เส้นประ)
    plt.scatter(X, y, color='navy', s=30, marker='o', label="Training Data")
    plt.plot(X_test, true_function(X_test), color='gray', linestyle='--', label="True Function")

    # พล็อตเส้นที่โมเดลทำนายได้ (เส้นทึบ)
    plt.plot(X_test, y_pred, color='red', label="Model Prediction")

    plt.xlabel("x (Features)")
    plt.ylabel("y (Target)")
    plt.xlim((0, 1))
    plt.ylim((-2, 2))
    plt.legend(loc="best")

    # กำหนดชื่อกราฟตาม Degree และอธิบายภาวะที่เกิดขึ้น
    if degree == 1:
        plt.title(f"Degree {degree}Underfitting (High Bias)")
    elif degree == 4:
        plt.title(f"Degree {degree}Sweet Spot (Good Generalization)")
    else:
        plt.title(f"Degree {degree}Overfitting (High Variance)")

plt.tight_layout()
plt.show()
```

ผลลัพธ์:



1772565623677

การวิเคราะห์ผลลัพธ์จากโค้ด (Visualizing the Tradeoff):

1. **Degree 1 (Underfitting - High Bias):** กราฟทางซ้ายสุดแสดงให้เห็นว่า การใช้สมการเส้นตรง (Degree 1) พยายามอธิบายข้อมูลที่มีลักษณะเป็นเส้นโค้งนั้นไม่เพียงพอ โมเดลมีข้อสมมติฐานที่เรียบง่ายเกินไป (High Bias) ทำให้เส้นทำนาย (สีแดง) ไม่สามารถสะท้อนความสัมพันธ์ที่แท้จริงของข้อมูลได้เลย เกิดความคลาดเคลื่อนสูงทั้งบริเวณตรงกลางและขอบของกราฟ
2. **Degree 4 (Sweet Spot - Good Generalization):** กราฟตรงกลางแสดงให้เห็นถึงจุดสมดุล โมเดลสมการกำลังสี่มีความยืดหยุ่นเพียงพอที่จะโค้งงอตามรูปแบบหลักของข้อมูลได้อย่างใกล้เคียงกับฟังก์ชันที่แท้จริง (เส้นประสีเทา) โดยไม่ได้พยายามลากผ่านทุกจุดข้อมูลที่มีสัญญาณรบกวนปะปนอยู่ โมเดลนี้จะสามารถทำนายข้อมูลใหม่ได้อย่างมีประสิทธิภาพ
3. **Degree 15 (Overfitting - High Variance):** กราฟทางขวาสุดแสดงให้เห็นอันตรายของการใช้โมเดลที่ซับซ้อนเกินไป สมการกำลังสิบห้ามีความยืดหยุ่นสูงมากจนสามารถลากเส้นผ่านจุดข้อมูลฝึกสอนได้เกือบทุกจุด (Training Error เข้าใกล้ 0) แต่การทำเช่นนั้นทำให้เส้นทำนายมีลักษณะแกว่งไปมาอย่างรุนแรง (Oscillation) ตามสัญญาณรบกวน (High Variance) หากมีข้อมูลใหม่ (Test Data) ตกอยู่ในบริเวณที่กราฟแกว่งตัว โมเดลจะทำนายผิดพลาดอย่างมหาศาล

แนวทางปฏิบัติในการรับมือกับ Bias และ Variance

ในฐานะนักวิทยาศาสตร์ข้อมูล เมื่อประเมินโมเดลและพบปัญหา จำเป็นต้องทราบวิธีการปรับแต่ง (Tuning) ที่เหมาะสม (Grus, 2019):

เมื่อเผชิญกับ High Bias (Underfitting):

- เพิ่มความซับซ้อนของโมเดล (เช่น เปลี่ยนจาก Linear Regression เป็น Random Forest หรือเพิ่มจำนวน Layer ใน Neural Network)
- สร้างคุณลักษณะใหม่ (Feature Engineering) ที่ช่วยให้โมเดลเห็นมุมมองของข้อมูลมากขึ้น
- ลดระดับการควบคุมความซับซ้อน (Decrease Regularization)

เมื่อเผชิญกับ High Variance (Overfitting):

- เพิ่มปริมาณข้อมูลฝึกสอน (Collect more training data) เพื่อให้โมเดลเห็นรูปแบบทั่วไปมากขึ้นและลดการจดจำ Noise
- ลดจำนวนคุณลักษณะ (Feature Selection) ตัดตัวแปรที่ไม่สำคัญทิ้ง
- ใช้เทคนิค Regularization (เช่น L1/L2 Penalty, Dropout) เพื่อบังคับให้โมเดลเรียนรู้เฉพาะสิ่งที่สำคัญจริงๆ
- ใช้เทคนิค Cross-Validation ในขั้นตอนการประเมินผล เพื่อให้มั่นใจว่าโมเดลไม่ได้ท่องจำข้อมูลเพียงชุดเดียว

สรุปประเด็นสำคัญ

- **Prediction Error** ใน Machine Learning ประกอบด้วย Bias, Variance และ Irreducible Error
- **Underfitting (High Bias):** เกิดจากโมเดลมีความเรียบง่ายเกินไป ไม่สามารถเรียนรู้รูปแบบของข้อมูลได้ ทำให้เกิดความคลาดเคลื่อนสูงทั้งบนข้อมูลฝึกสอนและข้อมูลทดสอบ
- **Overfitting (High Variance):** เกิดจากโมเดลมีความซับซ้อนสูงเกินไป จนไปท่องจำสัญญาณรบกวนในข้อมูลฝึกสอน ทำให้ทำนายข้อมูลชุดฝึกสอนได้ดีเยี่ยม แต่ทำนายข้อมูลใหม่ได้แย่มาก
- **Bias-Variance Tradeoff:** คือสัจธรรมที่ว่าการลด Bias จะทำให้ Variance เพิ่มขึ้น และในทางกลับกัน เป้าหมายของ Data Scientist คือการหา **Sweet Spot** ที่ให้ค่าความคลาดเคลื่อนรวม (Test Error) ต่ำที่สุด
- **Generalization:** คือความสามารถของโมเดลในการนำไปใช้งานจริงกับข้อมูลที่ไม่เคยเห็นมาก่อนได้อย่างแม่นยำ ซึ่งจะเกิดขึ้นเมื่อโมเดลอยู่ในจุดสมดุล

คำถามท้าย Section

1. หากท่านกำลังสร้างโมเดลทำนายโอกาสการเกิดโรคเบาหวานของผู้ป่วยในโรงพยาบาลศรีนครินทร์ และพบว่าโมเดลมีความแม่นยำบนข้อมูล Training set ถึง 99.5% แต่มีความแม่นยำบนข้อมูล Test set เพียง 65% โมเดลของท่านกำลังประสบภาวะใด และท่านมีแนวทางแก้ไขเบื้องต้นอย่างไร?
2. เพราะเหตุใดการนำข้อมูลทุกตัวแปร (Features) ที่หาได้ทั้งหมดมาใส่ในโมเดล Machine Learning จึงไม่ใช่แนวทางปฏิบัติที่ดีเสมอไป จงอธิบายโดยใช้หลักการ Bias-Variance Tradeoff

scikit-learn API

ไลบรารี scikit-learn เป็นหนึ่งในเครื่องมือที่ได้รับความนิยมสูงสุดสำหรับการพัฒนาระบบ Machine Learning ในภาษา Python ปัจจัยสำคัญที่ทำให้ scikit-learn ประสบความสำเร็จอย่างกว้างขวางในวงการวิทยาการข้อมูล ไม่ใช่เพียงเพราะมีอัลกอริทึมให้เลือกใช้งานจำนวนมาก แต่เป็นเพราะการออกแบบสถาปัตยกรรมซอฟต์แวร์ (Software Architecture) ที่มีชื่อเรียกว่า “Consistent Interface” หรือส่วนติดต่อการใช้งานที่มีความสม่ำเสมอ (Buitinck et al., 2013)

การออกแบบเชิงวัตถุ (Object-Oriented Design) ของ scikit-learn กำหนดให้ทุกอัลกอริทึม ไม่ว่าจะเป็นการทำความสะอาดข้อมูล (Data Imputation) การแปลงข้อมูล (Data Transformation) หรือการสร้างโมเดล (Modeling) ต้องปฏิบัติตามกฎเกณฑ์และเรียกใช้เมธอด (Method) ในรูปแบบเดียวกันทั้งหมด การทำความเข้าใจโครงสร้าง API ของ scikit-learn จึงเป็นรากฐานสำคัญที่ทำให้นักศึกษาสามารถเรียนรู้อัลกอริทึมใหม่ ๆ ได้อย่างรวดเร็ว และสามารถเขียนโค้ดที่ซับซ้อนได้อย่างเป็นระเบียบ (VanderPlas, 2016)

ในหัวข้อนี้ เราจะพิจารณาส่วนประกอบหลัก 3 ประการของ scikit-learn API ได้แก่ Estimator, Predictor และ Transformer รวมถึงการนำส่วนประกอบเหล่านี้มาประกอบเข้าด้วยกันผ่าน Pipeline

โครงสร้างพื้นฐาน: Estimator (ตัวประมาณค่า)

ใน scikit-learn วัตถุ (Object) ทุกชนิดที่สามารถเรียนรู้จากข้อมูลได้ จะถูกเรียกว่า **Estimator** ไม่ว่าวัตถุนั้นจะเป็นโมเดล Classification, Regression หรือกระบวนการจัดกลุ่ม (Clustering)

Estimator ทุกตัวจะมีกลไกการทำงานหลักผ่านเมธอด `fit()` ซึ่งทำหน้าที่ในการประมวลผลข้อมูลและเรียนรู้พารามิเตอร์จากชุดข้อมูลที่กำหนด (Training Data)

รูปแบบการเรียกใช้งาน:

1. **Unsupervised Learning:** `estimator.fit(X)` โดยที่ `X` คือเมทริกซ์ของฟีเจอร์ (Feature Matrix)
2. **Supervised Learning:** `estimator.fit(X, y)` โดยที่ `X` คือเมทริกซ์ของฟีเจอร์ และ `y` คือเวกเตอร์ของเป้าหมาย (Target Vector)

เมื่อเมธอด `fit()` ถูกเรียกใช้งาน Estimator จะทำการคำนวณและเก็บสถานะ (State) หรือพารามิเตอร์ที่เรียนรู้ได้ไว้ในตัวแปรภายใน (Internal Attributes) ของวัตถุนั้น ๆ ตามธรรมเนียมปฏิบัติของ scikit-learn ตัวแปรที่เกิดจากกระบวนการเรียนรู้จะมีเครื่องหมายขีดล่าง (Underscore) ต่อท้ายเสมอ เช่น `model.coef_` หรือ `model.intercept_`

ตัวอย่างในบริบทจริง: พิจารณาการประเมินราคาที่ดินในจังหวัดขอนแก่น หากเราใช้แบบจำลอง Linear Regression ตัว Estimator จะทำการเรียนรู้ความสัมพันธ์ระหว่างขนาดพื้นที่ (X) และราคาที่ดิน (y) ผ่านเมธอด `fit(X, y)` เมื่อกระบวนการเสร็จสิ้น ค่าน้ำหนัก (Weights) ที่โมเดลคำนวณได้จะถูกเก็บไว้ใน `coef_`

```python
from sklearn.linear_model import LinearRegression
import numpy as np

# สมมติฐานข้อมูล X: ขนาดที่ดิน (ตารางวา), y: ราคาประเมิน (ล้านบาท)
X_train = np.array([[50], [100], [150], [200]])
y_train = np.array([1.5, 3.2, 4.5, 5.8])

# 1. สร้าง Instance ของ Estimator
model = LinearRegression()

# 2. กระบวนการเรียนรู้ (Training)
model.fit(X_train, y_train)

# 3. ตรวจสอบพารามิเตอร์ที่เรียนรู้มาได้ (สังเกตเครื่องหมาย _)
print(f"ค่าน้ำหนัก (Coefficient): {model.coef_[0]:.4f}")
print(f"ค่าจุดตัดแกน Y (Intercept): {model.intercept_:.4f}")
```

ผลลัพธ์:

```
ค่าน้ำหนัก (Coefficient): 0.0284
ค่าจุดตัดแกน Y (Intercept): 0.2000
```

การทำนายและประเมินผล: Predictor (ตัวทำนาย)

เมื่อ Estimator ผ่านกระบวนการเรียนรู้ด้วยเมธอด `fit()` แล้ว หาก Estimator นั้นเป็นโมเดลที่สามารถทำการทำนายผลลัพธ์ได้ จะถูกจัดอยู่ในกลุ่มของ **Predictor** ซึ่งจะมีเมธอดสำคัญเพิ่มเติม ได้แก่ `predict()` และ `score()`

- `predict(X)`: ใช้สำหรับทำนายผลลัพธ์จากข้อมูลชุดใหม่ `X`
- `predict_proba(X)`: (เฉพาะบางโมเดล) ใช้สำหรับทำนายค่าความน่าจะเป็น (Probability) ของแต่ละคลาส ซึ่งมีประโยชน์อย่างมากในการประเมินความมั่นใจของโมเดล
- `score(X, y)`: ใช้สำหรับประเมินประสิทธิภาพของโมเดลเบื้องต้น โดยจะคืนค่าเป็นมาตรวัด (Metric) ที่เหมาะสมกับประเภทของโมเดล เช่น ค่า Accuracy สำหรับ Classification หรือค่า R^2 สำหรับ Regression

ตัวอย่างในบริบทจริง: ในระบบอนุมัติสินเชื่อของธนาคารพาณิชย์ หลังจากที่โมเดล (Predictor) เรียนรู้ประวัติการชำระหนี้ของลูกค้าแล้ว เมื่อมีลูกค้ารายใหม่ยื่นขอสินเชื่อ (`X_new`) ระบบจะใช้เมธอด `predict(X_new)` เพื่อตัดสินใจว่าควรอนุมัติหรือปฏิเสธ และอาจใช้ `predict_proba(X_new)` เพื่อดูว่ามีความเสี่ยงในการผิดนัดชำระหนี้กี่เปอร์เซ็นต์

เมธอด	หน้าที่	ผลลัพธ์ที่ได้
`fit(X, y)`	เรียนรู้จากข้อมูล	อัปเดตพารามิเตอร์ภายในโมเดล (`_`)
`predict(X)`	ทำนายผล	เวกเตอร์ของผลลัพธ์ (เช่น `[0, 1, 1, 0]`)
`predict_proba(X)`	ทำนายความน่าจะเป็น	เมทริกซ์ความน่าจะเป็นของแต่ละคลาส
`score(X, y)`	ประเมินความแม่นยำ	ตัวเลขทศนิยม (เช่น `0.95` หรือ 95%)

การแปลงข้อมูล: Transformer (ตัวแปลงข้อมูล)

ข้อมูลในโลกความเป็นจริงมักไม่พร้อมสำหรับการนำไปสร้างโมเดลโดยทันที อาจมีค่าสูญหาย (Missing Values) หรือมีสเกลที่แตกต่างกันมาก scikit-learn จัดการกับปัญหานี้ผ่านวัตถุประเภท **Transformer**

Transformer คือ Estimator ชนิดหนึ่งที่นอกจากจะมีเมธอด `fit()` เพื่อเรียนรู้ค่าทางสถิติจากข้อมูลแล้ว ยังมีเมธอด `transform()` เพื่อแปลงข้อมูลชุดนั้น (หรือชุดอื่น) ให้อยู่ในรูปแบบที่ต้องการ

- `fit(X)`: เรียนรู้ค่าทางสถิติ เช่น ค่าเฉลี่ย (Mean) หรือค่าเบี่ยงเบนมาตรฐาน (Standard Deviation) จากข้อมูล `X`
- `transform(X)`: นำค่าทางสถิติที่เรียนรู้มา ไปประยุกต์ใช้เพื่อแปลงข้อมูล `X`
- `fit_transform(X)`: เป็นการรวม `fit()` และ `transform()` เข้าด้วยกัน ซึ่งมักจะทำงานได้เร็วกว่าการเรียกแยกกัน

ข้อควรระวังที่สำคัญ (Data Leakage): กฎเหล็กในการใช้ Transformer คือ **ห้ามใช้ `fit()` หรือ `fit_transform()` กับข้อมูลชุดทดสอบ (Test Data) เด็ดขาด** เราจะต้องใช้ `fit_transform()` กับชุดข้อมูลฝึกสอน (Training Data) เท่านั้น จากนั้นจึงใช้เพียง `transform()` กับชุดข้อมูลทดสอบ การละเมิดกฎนี้จะทำให้เกิดปัญหา “ข้อมูลรั่วไหล” (Data Leakage) ซึ่งหมายถึงการที่โมเดลแอบรู้ข้อมูลทางสถิติของชุดทดสอบล่วงหน้า ทำให้ผลการประเมินความแม่นยำสูงเกินความเป็นจริง (James et al., 2023)

ตัวอย่างในบริบทจริง: พิจารณาการสร้างระบบวิเคราะห์ข้อมูลสุขภาพของโรงพยาบาล หากเราต้องการปรับสเกล (Scaling) ข้อมูลความดันโลหิตและระดับน้ำตาลในเลือดให้อยู่ในมาตรฐานเดียวกันด้วย `StandardScaler`

```python
from sklearn.preprocessing import StandardScaler
import numpy as np

# X_train: ข้อมูลผู้ป่วยชุดฝึกสอน, X_test: ข้อมูลผู้ป่วยชุดทดสอบ
X_train = np.array([[120, 95], [130, 105], [125, 100]])
X_test = np.array([[140, 110], [115, 90]])

scaler = StandardScaler()

# ถูกต้อง: เรียนรู้ค่าเฉลี่ยจาก X_train และแปลง X_train
X_train_scaled = scaler.fit_transform(X_train)

# ถูกต้อง: แปลง X_test โดยใช้ค่าเฉลี่ยที่เรียนรู้มาจาก X_train เท่านั้น
X_test_scaled = scaler.transform(X_test)

# ผิดอย่างร้ายแรง: scaler.fit_transform(X_test) → ห้ามทำเด็ดขาด!
```

> **หมายเหตุ:** โค้ดด้านบนเป็นตัวอย่างเชิงโครงสร้างที่แสดงวิธีการใช้ Transformer อย่างถูกต้อง ไม่มี print output แต่ `X_train_scaled` และ `X_test_scaled` จะเป็น array ที่ปรับสเกลแล้ว พร้อมนำไปใช้กับโมเดลในขั้นตอนถัดไป

การเชื่อมต่อกระบวนการ: Pipeline (ท่อส่งข้อมูล)

ในการทำงานจริง กระบวนการทำ Machine Learning มักประกอบด้วยขั้นตอนย่อยหลายขั้นตอน เช่น การเติมค่าสูญหาย → การปรับสเกลข้อมูล → การสร้างโมเดล หากเราเขียนโค้ดแยกแต่ละขั้นตอน ตัวแปรจะมีความซับซ้อนและเสี่ยงต่อการเกิดข้อผิดพลาด (เช่น การลืม `transform` ข้อมูลชุดทดสอบ)

scikit-learn แก้ปัญหานี้ด้วยคลาส **`Pipeline`** ซึ่งทำหน้าที่ร้อยเรียง Transformer หลาย ๆ ตัว และปิดท้ายด้วย Estimator/Predictor หนึ่งตัว ให้อยู่ในรูปแบบของวัตถุเดียว (Single Object) (McKinney, 2022)

แผนภาพแสดงการทำงานของ Pipeline:

```
ข้อมูลเข้า (X)
   |
   ▼
┌──────────────────────────────────────────┐
| Pipeline                                 |
|                                          |
| 1. [Transformer] SimpleImputer           |──> เติมค่าที่หายไป
|          |                               |
|          ▼                               |
| 2. [Transformer] StandardScaler          |──> ปรับสเกลข้อมูล
|          |                               |
|          ▼                               |
| 3. [Predictor]   LogisticRegression      |──> ทำนายผลลัพธ์
└──────────────────────────────────────────┘
   |
   ▼
ผลลัพธ์ (Predictions / Score)
```

ข้อดีของ Pipeline คือเมื่อเราเรียกใช้ `pipeline.fit(X_train, y_train)` ระบบจะทำการ `fit_transform` ข้อมูลผ่าน Transformer ทุกตัวตามลำดับ และนำผลลัพธ์สุดท้ายไป `fit` กับโมเดลให้โดยอัตโนมัติ และเมื่อเราเรียก `pipeline.predict(X_test)` ระบบก็จะทำการ `transform` ข้อมูลชุดทดสอบตามลำดับอย่างถูกต้อง ป้องกันปัญหา Data Leakage ได้ 100%

ตัวอย่างการใช้งานจริงแบบ End-to-End: ชุดข้อมูล Titanic

เพื่อให้เห็นภาพการทำงานร่วมกันของ Estimator, Transformer และ Pipeline อย่างสมบูรณ์ เราจะพิจารณาชุดข้อมูลคลาสสิกอย่าง Titanic ซึ่งเป้าหมายคือการทำนายว่าผู้โดยสารจะรอดชีวิตหรือไม่ (Classification Problem) จากคุณลักษณะต่าง ๆ เช่น อายุ เพศ และระดับชั้นของตั๋ว

ในตัวอย่างนี้ เราจะจำลองสถานการณ์ที่มีทั้งข้อมูลเชิงตัวเลข (Numeric) และข้อมูลเชิงกลุ่ม (Categorical) ซึ่งต้องใช้วิธีการ Preprocessing ที่แตกต่างกัน เราจึงจะใช้ `ColumnTransformer` เข้ามาช่วยแยกสายการประมวลผล

```python
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

# 1. โหลดข้อมูล Titanic จาก OpenML
print("กำลังโหลดข้อมูล...")
titanic = fetch_openml('titanic', version=1, as_frame=True, parser='auto')
df = titanic.frame

# เลือกเฉพาะฟีเจอร์ที่สนใจเพื่อความเข้าใจง่าย
features = ['pclass', 'sex', 'age', 'fare']
X = df[features]
y = df['survived'].astype(int)

# 2. แบ่งข้อมูลเป็นชุดฝึกสอนและชุดทดสอบ
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. กำหนดกลุ่มของฟีเจอร์
numeric_features = ['age', 'fare']
categorical_features = ['pclass', 'sex']

# 4. สร้าง Transformer สำหรับข้อมูลตัวเลข (เติมค่าเฉลี่ย → ปรับสเกล)
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

# 5. สร้าง Transformer สำหรับข้อมูลกลุ่ม (เติมค่าที่พบบ่อย → แปลงเป็น One-Hot)
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# 6. รวม Transformer ทั้งสองสายเข้าด้วยกันด้วย ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# 7. สร้าง Pipeline หลักที่รวมการ Preprocess และ Model เข้าด้วยกัน
# โมเดลที่ใช้คือ Random Forest (Predictor)
clf_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
])

# 8. กระบวนการเรียนรู้ (Training)
# การเรียก fit เพียงครั้งเดียว จะสั่งรันกระบวนการทั้งหมดใน Pipeline
clf_pipeline.fit(X_train, y_train)

# 9. การประเมินผล (Evaluation)
# การเรียก score จะทำการ transform X_test อัตโนมัติ ป้องกัน Data Leakage
train_score = clf_pipeline.score(X_train, y_train)
test_score = clf_pipeline.score(X_test, y_test)

print(f"ความแม่นยำบนชุดฝึกสอน (Training Accuracy): {train_score:.4f}")
print(f"ความแม่นยำบนชุดทดสอบ (Test Accuracy): {test_score:.4f}")

# 10. การนำไปใช้งานจริง (Prediction)
# สมมติมีผู้โดยสารใหม่ 1 คน
new_passenger = pd.DataFrame({
    'pclass': [3], 'sex': ['male'], 'age': [25.0], 'fare': [7.5]
})
prediction = clf_pipeline.predict(new_passenger)
prob = clf_pipeline.predict_proba(new_passenger)

print(f"ผลการทำนายผู้โดยสารใหม่: {'รอดชีวิต' if prediction[0] == 1 else 'เสียชีวิต'}")
print(f"ความน่าจะเป็นที่จะรอดชีวิต: {prob[0][1]*100:.2f}%")
```

ผลลัพธ์:

```
กำลังโหลดข้อมูล...

ความแม่นยำบนชุดฝึกสอน (Training Accuracy): 0.9808
ความแม่นยำบนชุดทดสอบ (Test Accuracy): 0.7862

ผลการทำนายผู้โดยสารใหม่: เสียชีวิต
ความน่าจะเป็นที่จะรอดชีวิต: 12.50%
```

จากการศึกษาโค้ดข้างต้น จะเห็นได้ว่าการใช้ Pipeline ร่วมกับ Consistent API ของ scikit-learn ทำให้โค้ดมีความเป็นระเบียบ อ่านง่าย และลดโอกาสในการเกิดข้อบกพร่องทางตรรกะ (Logical Errors) ที่มักเกิดขึ้นในขั้นตอนการจัดการข้อมูลได้อย่างมีประสิทธิภาพ (Knaflic, 2015)

สรุปประเด็นสำคัญ

- **Consistent Interface:** scikit-learn ออกแบบให้ทุกอัลกอริทึมใช้งานผ่านรูปแบบเดียวกัน ทำให้ง่ายต่อการเรียนรู้และนำไปประยุกต์ใช้
- **Estimator และ Predictor:** ใช้เมธอด `fit()` เพื่อเรียนรู้จากข้อมูล และใช้ `predict()` หรือ `score()` เพื่อทำนายและประเมินผล
- **Transformer:** ใช้สำหรับแปลงข้อมูล โดยต้องระวังการเกิด Data Leakage ซึ่งป้องกันได้โดยการใช้ `fit_transform()` บน Training Data และใช้เพียง `transform()` บน Test Data
- **Pipeline:** เป็นเครื่องมือสำคัญที่ใช้รวมขั้นตอนการแปลงข้อมูลและการสร้างโมเดลเข้าไว้ด้วยกัน ช่วยลดข้อผิดพลาดและทำให้โค้ดเป็นระเบียบ

คำถามท้าย Section

1. หากนักศึกษาเผลอใช้คำสั่ง `scaler.fit_transform(X_test)` ก่อนที่จะทำการประเมินโมเดลด้วย `model.score(X_test, y_test)` จะส่งผลกระทบต่อความน่าเชื่อถือของผลการประเมินอย่างไร? จงอธิบายโดยใช้หลักการของ Data Leakage
2. ในกรณีที่ชุดข้อมูลมีตัวแปรอิสระ (Features) ประกอบด้วย ข้อความ (Text), รูปภาพ (Images), และตัวเลข (Numeric) นักศึกษาจะประยุกต์ใช้ `ColumnTransformer` และ `Pipeline` เพื่อสร้างสถาปัตยกรรมข้อมูล (Data Pipeline) อย่างไร?

LLM ในกระบวนการ Data Science

โมเดลภาษาขนาดใหญ่ (Large Language Models: LLMs) เช่น GPT-4, Claude หรือ Gemini ได้สร้างการเปลี่ยนแปลงครั้งสำคัญในกระบวนทัศน์ของการพัฒนาซอฟต์แวร์และการวิเคราะห์ข้อมูล ในบริบทของวิทยาการข้อมูล (Data Science) LLM ไม่ได้เข้ามาทดแทนนักวิทยาศาสตร์ข้อมูล แต่ทำหน้าที่เป็น "ผู้ช่วยอัจฉริยะ" (AI Assistant หรือ Copilot) ที่ช่วยเพิ่มประสิทธิภาพ (Productivity) ลดระยะเวลาในการเขียนโค้ดพื้นฐาน และช่วยขยายมุมมองในการวิเคราะห์ข้อมูล การทำความเข้าใจวิธีการทำงานร่วมกับ LLM อย่างมีประสิทธิภาพและตระหนักถึงข้อจำกัด จึงเป็นทักษะที่จำเป็นสำหรับนักวิทยาศาสตร์ข้อมูลในยุคปัจจุบัน

บทบาทของ LLM ในฐานะผู้ช่วยนักวิทยาศาสตร์ข้อมูล

กระบวนการวิทยาการข้อมูลประกอบด้วยหลายขั้นตอน ตั้งแต่การทำความเข้าใจปัญหา การเตรียมข้อมูล การสร้างโมเดล ไปจนถึงการสื่อสารผลลัพธ์ LLM สามารถเข้ามามีส่วนร่วมในแต่ละขั้นตอนได้ดังต่อไปนี้

การสำรวจและวิเคราะห์ข้อมูลเบื้องต้น (EDA Suggestions)

บ่อยครั้งที่นักวิทยาศาสตร์ข้อมูลต้องเผชิญกับชุดข้อมูลใหม่และต้องการแนวทางในการเริ่มต้นวิเคราะห์ (Cold Start Problem) LLM สามารถทำหน้าที่เป็นคู่คิดในการระดมสมอง (Brainstorming) เพื่อเสนอแนะสมมติฐานหรือมุมมองที่ควรตรวจสอบ

ตัวอย่าง Prompt สำหรับ EDA:

> "ทำหน้าที่เป็นนักวิทยาศาสตร์ข้อมูลอาวุโส ฉันมีชุดข้อมูลประวัติการรักษาผู้ป่วยโรคเบาหวานของโรงพยาบาลในจังหวัดขอนแก่น ประกอบด้วยคอลัมน์: อายุ, เพศ, ระดับน้ำตาลในเลือด, ความดันโลหิต, ประวัติครอบครัว, และการกลับมาแอดมิทซ้ำภายใน 30 วัน (Target) จงเสนอแนะแนวทางในการทำ Exploratory Data Analysis (EDA) จำนวน 5 ข้อ พร้อมระบุประเภทของกราฟที่เหมาะสมสำหรับการวิเคราะห์แต่ละข้อ"

การสร้างโค้ด (Code Generation)

การเขียนโค้ดสำหรับการจัดการข้อมูล (Data Manipulation) หรือการสร้างโมเดล Machine Learning มักมีรูปแบบที่ตายตัว (Boilerplate code) LLM มีความสามารถสูงในการสร้างโค้ด Python โดยใช้ไลบรารีมาตรฐาน เช่น `pandas`, `scikit-learn` หรือ `matplotlib` ตามคำสั่งที่เป็นภาษาธรรมชาติ

ตัวอย่าง Prompt สำหรับ Code Generation:

> "เขียนโค้ด Python โดยใช้ไลบรารี pandas และ scikit-learn เพื่ออ่านไฟล์ `customers.csv` ทำการจัดการค่าสูญหาย (Missing values) ในคอลัมน์ 'Income' ด้วยค่ามัธยฐาน และสร้างโมเดล Random Forest Classifier เพื่อทำนายคอลัมน์ 'Churn' พร้อมทั้งแสดงค่า Accuracy"

```python
# นำเข้าไลบรารีที่จำเป็น
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.impute import SimpleImputer

# 1. โหลดข้อมูลลูกค้าธนาคาร (สมมติว่าเป็นข้อมูลลูกค้าในประเทศไทย)
# หมายเหตุ: ในการใช้งานจริงต้องตรวจสอบ path ของไฟล์ให้ถูกต้อง
df = pd.pd.read_csv('customers.csv')

# 2. แยก Features (X) และ Target (y)
X = df.drop('Churn', axis=1)
y = df['Churn']

# 3. จัดการค่าสูญหายในคอลัมน์ Income ด้วยค่ามัธยฐาน (Median)
imputer = SimpleImputer(strategy='median')
# สมมติว่า Income เป็นคอลัมน์ที่ 2 (index 1) หรือระบุชื่อคอลัมน์โดยตรง
X['Income'] = imputer.fit_transform(X[['Income']])

# 4. แบ่งข้อมูลเป็นชุดฝึกสอน (Training set) และชุดทดสอบ (Test set)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. สร้างและฝึกสอนโมเดล Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# 6. ทำนายผลและประเมินความแม่นยำ
predictions = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print(f"ความแม่นยำของโมเดล (Accuracy): {accuracy:.4f}")
```

> **หมายเหตุ:** โค้ดด้านบนเป็นตัวอย่างผลลัพธ์จาก LLM ที่ใช้เพื่อประกอบการอธิบาย และมีข้อผิดพลาดที่รอให้นักศึกษาตรวจสอบ เช่น `pd.pd.read_csv` ควรเป็น `pd.read_csv` ซึ่งเป็นตัวอย่างที่ดีว่า **ต้องตรวจสอบโค้ดจาก LLM เสมอ**

การแก้ไขข้อผิดพลาดของโค้ด (Debugging)

เมื่อเกิดข้อผิดพลาดในการรันโปรแกรม (Runtime Error) ข้อความแจ้งข้อผิดพลาด (Stack Trace) มักมีความซับซ้อน LLM สามารถอธิบายสาเหตุของข้อผิดพลาดและเสนอวิธีการแก้ไขได้อย่างแม่นยำ

ตัวอย่าง Prompt สำหรับ Debugging:

> "ฉันรันโค้ดเพื่อฝึกสอนโมเดล Logistic Regression แต่พบข้อผิดพลาด `ValueError: Input contains NaN, infinity or a value too large for dtype('float64')` ข้อผิดพลาดนี้เกิดจากอะไร และฉันจะใช้ pandas ตรวจสอบและแก้ไขปัญหานี้ได้อย่างไร"

การตีความผลลัพธ์และการเขียนรายงาน (Interpretation and Report Writing)

การสื่อสารผลลัพธ์ทางสถิติให้ผู้มีส่วนได้ส่วนเสีย (Stakeholders) ทางธุรกิจเข้าใจ เป็นความท้าทายหลักของนักวิทยาศาสตร์ข้อมูล LLM สามารถช่วยแปลความหมายของตัวชี้วัด (Metrics) ทางเทคนิคให้เป็นภาษาธุรกิจที่เข้าใจง่าย (Knaflic, 2015)

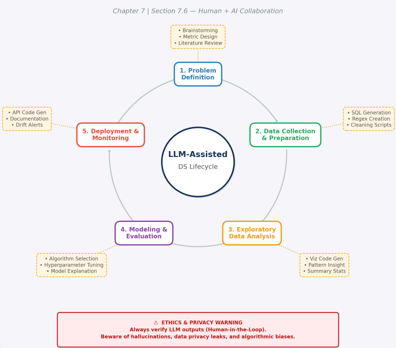
ตัวอย่าง Prompt สำหรับการตีความ:

> “โมเดลทำนายการลาออกของพนักงานมหาวิทยาลัยมีค่า Recall สำหรับคลาส ‘ลาออก’ อยู่ที่ 0.90 และค่า Precision อยู่ที่ 0.60 จงอธิบายความหมายของตัวเลขสองค่านี้ให้ผู้บริหารฝ่ายทรัพยากรบุคคล (HR) ฟัง โดยเน้นถึงผลกระทบทางธุรกิจและข้อควรระวังในการนำโมเดลไปใช้”

สถาปัตยกรรมการทำงานร่วมกันระหว่างมนุษย์และ LLM

การใช้งาน LLM อย่างมีประสิทธิภาพในงานวิทยาการข้อมูล ควรมีลักษณะเป็นวงรอบของการทำงานร่วมกัน (Human-in-the-loop) ดังแผนภาพด้านล่าง

```
+------------------------+            (1) กำหนดบริบทและ Prompt          +------------------------+

|                        | ---------------------------------------------> |                        |
|  นักวิทยาศาสตร์ข้อมูล       |                                              |  Large Language Model   |
|  (Data Scientist)      | <--------------------------------------------- |  (LLM)                 |
|                        |  (2) วิเคราะห์, สร้าง Code, หรือให้ข้อเสนอแนะ         |                        |
+------------------------+                                                +------------------------+
          |  ^
          |  | (4) ปรับปรุง Prompt / แก้ไข Error
          v  |
(3) ทดสอบการทำงาน (Execution)
    - รันโค้ด Python
    - ตรวจสอบความถูกต้องของข้อมูล
    - ประเมินผลลัพธ์ทางสถิติ
```



LLM-Assisted Data Science Workflow

แผนภาพที่ 7.6.1: วงรอบการทำงานร่วมกันระหว่างมนุษย์และ LLM ในกระบวนการ Data Science

---

ข้อจำกัดและความเสี่ยงของ LLM (Limitations and Risks)

แม้ LLM จะมีประสิทธิภาพสูง แต่การพึ่งพาโมเดลเหล่านี้โดยปราศจากการตรวจสอบเชิงวิพากษ์ (Critical Evaluation) อาจนำไปสู่ความผิดพลาดร้ายแรงได้ นักศึกษาต้องตระหนักถึงข้อจำกัดต่อไปนี้:

1. **อาการประสาทหลอน (Hallucination):** LLM ทำงานโดยการทำนายคำถัดไปตามความน่าจะเป็นทางสถิติ ไม่ได้มีความเข้าใจตรรกะอย่างแท้จริง บ่อยครั้งที่ LLM สร้างข้อมูลอ้างอิงปลอม หรือสร้างชื่อฟังก์ชันในไลบรารีที่ไม่มีอยู่จริง (ตัวอย่างเช่น การแนะนำให้ใช้ `sklearn.ensemble.SuperFastTree` ซึ่งไม่มีอยู่จริงใน Scikit-learn)
2. **ความผิดพลาดเชิงตรรกะทางคณิตศาสตร์ (Mathematical Inaccuracy):** LLM ไม่ใช่เครื่องคิดเลข หากให้ LLM คำนวณค่าสถิติซับซ้อนจากชุดข้อมูลโดยตรง (โดยไม่ผ่านการเขียนและรันโค้ด Python) ผลลัพธ์ที่ได้มักจะคลาดเคลื่อน นักวิทยาศาสตร์ข้อมูลต้องใช้ LLM เพื่อสร้างโค้ดสำหรับการคำนวณ ไม่ใช่ให้ LLM รับหน้าที่คำนวณเสียเอง
3. **ข้อมูลที่ล้าสมัย (Knowledge Cutoff):** LLM ถูกฝึกสอนด้วยข้อมูลที่มีจุดสิ้นสุด (Cutoff date) หากมีไลบรารีเวอร์ชันใหม่ที่มีการเปลี่ยนชื่อฟังก์ชัน (Deprecation) LLM อาจยังคงแนะนำให้ใช้โค้ดเวอร์ชันเก่าที่รันไม่ผ่านในปัจจุบัน

ความสามารถ	สิ่งที่ LLM ทำได้ดี (ควรใช้)	สิ่งที่ LLM ทำได้ไม่ดี (ควรหลีกเลี่ยง / ต้องตรวจสอบ)
การเขียนโค้ด	สร้างโครงร่างโค้ด (Boilerplate), เขียน Regular Expression	โค้ดที่ต้องการการปรับแต่งประสิทธิภาพขั้นสูง (Optimization)
การคำนวณ	เขียนโค้ดคณิตศาสตร์ให้ Python ไปรัน	คำนวณค่าสถิติจากข้อมูลดิบโดยตรงผ่าน Prompt
ความรู้	อธิบายทฤษฎี Machine Learning พื้นฐาน	อ้างอิงเอกสารวิชาการ (มักสร้าง Citation ปลอม)
ข้อมูล	สร้างข้อมูลจำลอง (Dummy data) เพื่อทดสอบ	นำเข้าข้อมูลส่วนบุคคล (PDPA) เพื่อให้ LLM วิเคราะห์

ตารางที่ 7.6.1: เปรียบเทียบความเหมาะสมในการใช้งาน LLM สำหรับงาน Data Science

---

จริยธรรมและความรับผิดชอบทางวิชาการ (Ethics and Academic Integrity)

การใช้ LLM ในการเรียนและการทำงานจริงเกี่ยวข้องกับประเด็นทางจริยธรรมที่สำคัญ ซึ่งนักศึกษาสาขาวิทยาการคอมพิวเตอร์ต้องยึดถืออย่างเคร่งครัด:

หลักการ “ห้ามคัดลอกโดยปราศจากความเข้าใจ” (No Blind Copying)

การนำโค้ดที่สร้างโดย LLM ไปใช้งานโดยไม่เข้าใจการทำงานของแต่ละบรรทัด ถือเป็นความเสี่ยงขั้นร้ายแรง หากเกิดข้อผิดพลาดในการทำนาย (เช่น โมเดลอนุมัติสินเชื่อปฏิเสธลูกค้าอย่างไม่เป็นธรรม) นักวิทยาศาสตร์ข้อมูลผู้เขียนโปรแกรมจะต้องเป็นผู้รับผิดชอบ ไม่ใช่ AI ดังนั้น **“คุณต้องสามารถอธิบายโค้ดทุกบรรทัดที่คุณนำไปใช้งานได้”**

ความเป็นส่วนตัวและความปลอดภัยของข้อมูล (Data Privacy and Security)

ห้ามคัดลอกข้อมูลจริงที่มีความละเอียดอ่อน (Sensitive Data) เช่น ข้อมูลส่วนบุคคลของลูกค้า ข้อมูลเวชระเบียนผู้ป่วย หรือข้อมูลทางการเงินขององค์กร นำไปป้อนลงใน Prompt ของ LLM สาธารณะ (Public LLM) อย่างเด็ดขาด เนื่องจากข้อมูลเหล่านี้อาจถูกนำไปใช้ฝึกสอนโมเดลต่อและละเมิดพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล (PDPA) หากต้องการให้ LLM ช่วยวิเคราะห์ ให้ป้อนเฉพาะโครงสร้างข้อมูล (Schema) หรือข้อมูลจำลอง (Mock Data) เท่านั้น

ความโปร่งใสและการอ้างอิง (Transparency and Citation)

ในบริบททางวิชาการ เช่น การทำโครงงาน (Project) หรือการเขียนรายงาน หากมีการใช้ LLM ช่วยในการเขียนโค้ดหรือปรับปรุงภาษา นักศึกษาต้องแสดงความโปร่งใสโดยระบุขอบเขตการใช้งานอย่างชัดเจน

ตัวอย่างการอ้างอิงการใช้งาน LLM ในรายงาน:

> *ข้อความอ้างอิง:* “ในการพัฒนาโครงงานนี้ ผู้วิจัยได้ใช้ ChatGPT (รุ่น GPT-4, OpenAI) ในขั้นตอนการทำ Exploratory Data Analysis เพื่อระดมสมองในการตั้งสมมติฐาน และใช้ในการช่วยเขียนโครงร่างโค้ดสำหรับการพล็อตกราฟด้วยไลบรารี Seaborn โดยผู้วิจัยได้ทำการตรวจสอบความถูกต้องและปรับปรุงโค้ดทั้งหมดด้วยตนเอง”

การใช้งาน LLM ในกระบวนการ Data Science ไม่ใช่การลดทอนคุณค่าของทักษะการเขียนโปรแกรม แต่เป็นการยกระดับกระบวนการทำงานให้รวดเร็วและกว้างขวางขึ้น ผู้ที่จะประสบความสำเร็จในสายงานนี้ คือผู้ที่มีรากฐานทางคณิตศาสตร์ สถิติ และการเขียนโปรแกรมที่แข็งแรงพอที่จะตรวจสอบ (Verify) และต่อยอด (Extend) ผลลัพธ์ที่ AI สร้างขึ้นได้ (VanderPlas, 2016; McKinney, 2022)

สรุปประเด็นสำคัญ

- **บทบาทของ LLM:** ทำหน้าที่เป็นผู้ช่วย (Copilot) ในการเสนอแนะมุมมอง EDA, สร้างโครงร่างโค้ด (Boilerplate), ช่วยหาข้อผิดพลาด (Debugging) และแปลความหมายทางสถิติเป็นภาษาธุรกิจ
- **ข้อจำกัดที่ต้องระวัง:** LLM อาจเกิดอาการประสาทหลอน (Hallucination) โดยการสร้างฟังก์ชันที่ไม่มีอยู่จริง และไม่สามารถคำนวณทางคณิตศาสตร์ที่ซับซ้อนได้อย่างแม่นยำ
- **ความรับผิดชอบของมนุษย์:** นักวิทยาศาสตร์ข้อมูลต้องเป็นผู้ตรวจสอบ (Verify) ความถูกต้องของโค้ดและผลลัพธ์เสมอ ห้ามนำโค้ดไปใช้โดยปราศจากความเข้าใจ
- **ความปลอดภัยของข้อมูล:** ห้ามป้อนข้อมูลส่วนบุคคลหรือข้อมูลลับขององค์กร (Sensitive Data) ลงใน LLM สาธารณะโดยเด็ดขาด เพื่อป้องกันการละเมิด PDPA
- **ความโปร่งใสทางวิชาการ:** หากใช้ LLM ในการทำโครงงานหรือรายงาน ต้องมีการอ้างอิงและระบุขอบเขตการใช้งานอย่างชัดเจน

คำถามท้าย Section

1. หากนักศึกษาได้รับมอบหมายให้วิเคราะห์ชุดข้อมูลประวัติการกู้ยืมเงินของนักศึกษามหาวิทยาลัยขอนแก่น นักศึกษาจะประยุกต์ใช้ LLM ในขั้นตอน EDA อย่างไรให้ปลอดภัยและไม่ละเมิดความเป็นส่วนตัวของข้อมูล?
2. อาการประสาทหลอน (Hallucination) ของ LLM ในบริบทของการเขียนโปรแกรม Machine Learning มีลักษณะอย่างไร จงยกตัวอย่างสถานการณ์ที่อาจก่อให้เกิดความเสียหายหากนักศึกษาไม่ตรวจสอบโค้ดให้ดี
3. ทำไมการให้ LLM “เขียนโค้ด Python เพื่อคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน” จึงเป็นวิธีปฏิบัติที่ถูกต้องและเชื่อถือได้มากกว่าการ “คัดลอกตารางข้อมูลลงใน Prompt แล้วสั่งให้ LLM คำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานมาให้”?

สรุปท้ายบท

ในบทนี้ นักศึกษาได้เรียนรู้เกี่ยวกับ: - **Machine Learning:** เป็นสาขาย่อยของ AI ที่ให้คอมพิวเตอร์เรียนรู้จากข้อมูลโดยไม่ต้องเขียนกฎเกณฑ์ - **Supervised vs Unsupervised:** Supervised ใช้ Label, Unsupervised ค้นหา Pattern โดยไม่มี Label - **Bias-Variance Tradeoff:** โมเดลที่ดีต้องสมดุลระหว่าง Underfitting และ Overfitting - **scikit-learn:** API มาตรฐาน: fit() → predict() → score() ใช้ได้กับทุกอัลกอริทึม - **LLM ใน DS:** Large Language Models ช่วยเร่งกระบวนการ Data Science ได้ แต่ต้องตรวจสอบผลลัพธ์

คำศัพท์สำคัญ: ความแตกต่างระหว่าง Traditional Programming และ Machine Learning, ความสัมพันธ์ระหว่าง AI, ML, DL และ GenAI, การประยุกต์ใช้ Machine Learning ในชีวิตประจำวัน, Supervised Learning (การเรียนรู้แบบมีผู้สอน), Unsupervised Learning (การเรียนรู้แบบไม่มีผู้สอน), Semi-supervised Learning (การเรียนรู้แบบกึ่งมีผู้สอน), Reinforcement Learning (การเรียนรู้แบบเสริมกำลัง), ตารางสรุปประเภทของ Machine Learning

เอกสารอ้างอิง

- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., ... & Varoquaux, G. (2013). API design for machine learning software: experiences from the scikit-learn project. *arXiv preprint arXiv:1309.0238*.
- Grus, J. (2019). *Data Science from Scratch: First Principles with Python* (2nd ed.). O'Reilly Media.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python* (2nd ed.). Springer.
- Knaflic, C. N. (2015). *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). The MIT Press.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., ... & Wen, J. R. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*.

✏ แบบฝึกหัดท้ายบท

ส่วนที่ 1: คำถามทบทวน

คำชี้แจง: *จงเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว*

1 **สำนักบริหารและพัฒนาวิชาการ มหาวิทยาลัยขอนแก่น ต้องการสร้างโมเดลเพื่อทำนายว่านักศึกษาชั้นปีที่ 1 จะมีแนวโน้ม "พ้นสภาพการเป็นนักศึกษา (Dropout)" หรือ "ศึกษาต่อ (Retain)" โดยใช้ข้อมูลเกรดเฉลี่ย สถิติการเข้าเรียน และคะแนนสอบเข้า ปัญหาดังกล่าวจัดอยู่ใน Machine Learning ประเภทใด**

A Unsupervised Learning (Clustering)
B Supervised Learning (Classification)
C Supervised Learning (Regression)
D Reinforcement Learning

▶ แสดงเฉลย

2 **หากโมเดล Machine Learning มีประสิทธิภาพสูงมากเมื่อทดสอบกับชุดข้อมูลฝึกสอน (Training Data) แต่กลับมีประสิทธิภาพต่ำมากเมื่อนำไปใช้กับชุดข้อมูลทดสอบ (Testing Data) สถานการณ์นี้เรียกว่าอะไร และมีความสัมพันธ์กับ Bias-Variance อย่างไร**

A Underfitting เกิดจาก High Bias และ Low Variance
B Underfitting เกิดจาก Low Bias และ High Variance
C Overfitting เกิดจาก High Bias และ Low Variance
D Overfitting เกิดจาก Low Bias และ High Variance

▶ แสดงเฉลย

3 **ในการใช้งาน scikit-learn API ขั้นตอนใดคือการให้โมเดลเรียนรู้ความสัมพันธ์จากข้อมูลฝึกสอน (Training Data)**

A `model.predict(X_train)`
B `model.transform(X_train)`
C `model.fit(X_train, y_train)`
D `model.evaluate(X_train, y_train)`

▶ แสดงเฉลย

4 **ข้อใดคือเหตุผลสำคัญที่สุดในการแบ่งข้อมูลออกเป็นชุดข้อมูลฝึกสอน (Training Set) และชุดข้อมูลทดสอบ (Testing Set) ในกระบวนการ ML Workflow**

A เพื่อลดระยะเวลาในการประมวลผลของคอมพิวเตอร์
B เพื่อป้องกันปัญหา Data Leakage และประเมินความสามารถในการทำนายข้อมูลที่ไม่เคยเห็นมาก่อน (Generalization)
C เพื่อเพิ่มปริมาณข้อมูลให้โมเดลเรียนรู้ได้มากขึ้น
D เพื่อให้สามารถใช้ Large Language Models (LLMs) ในการสร้างโค้ดได้ง่ายขึ้น

▶ แสดงเฉลย

5 **ข้อใดคือการประยุกต์ใช้ Large Language Models (LLMs) ในกระบวนการ Data Science ที่ *ไม่เหมาะสม* และอาจก่อให้เกิดความเสี่ยงทางจริยธรรมหรือกฎหมาย**

A ใช้ LLMs ช่วยเขียน Regular Expression เพื่อทำความสะอาดข้อมูลข้อความ
B อัปโหลดฐานข้อมูลประวัติการรักษาและชื่อผู้ป่วยของโรงพยาบาลเข้าสู่ ChatGPT เพื่อให้ช่วยจัดกลุ่มผู้ป่วย
C ใช้ LLMs ช่วยอธิบายความหมายของ Error Message ที่เกิดขึ้นจากการรันโค้ด scikit-learn
D ใช้ LLMs ร่างโครงร่างรายงานสรุปผลการวิเคราะห์ข้อมูลเพื่อนำเสนอผู้บริหาร

▶ แสดงเฉลย

ส่วนที่ 2: คำถามอัตนัย

คำชี้แจง: *จงอธิบาย วิเคราะห์ หรือเปรียบเทียบ ตามประเด็นที่กำหนดให้ชัดเจน*

1 การวิเคราะห์ข้อมูลผู้ป่วยในโรงพยาบาล

โรงพยาบาลศรีนครินทร์มีฐานข้อมูลผู้ป่วยโรคเบาหวานจำนวน 10,000 ราย ประกอบด้วยข้อมูล อายุ น้ำหนัก ระดับน้ำตาลในเลือด และความดันโลหิต ให้นักศึกษาอธิบายว่า หากต้องการใช้ Machine Learning กับข้อมูลชุดนี้ การตั้งโจทย์แบบ Supervised Learning และ Unsupervised Learning จะมีความแตกต่างกันอย่างไร พร้อมยกตัวอย่างโจทย์วิจัยมาประเภทละ 1 ตัวอย่าง

▸ แสดงแนวคำตอบ

2 Bias-Variance Tradeoff

จงอธิบายแนวคิดของ Bias-Variance Tradeoff โดยเปรียบเทียบกับการเตรียมตัวสอบกลางภาควิชา Introduction to Data Science ของนักศึกษา พร้อมระบุว่าสถานการณ์ใดเปรียบเสมือน Overfitting และสถานการณ์ใดเปรียบเสมือน Underfitting

▸ แสดงแนวคำตอบ

3 บทบาทของ LLM ใน ML Workflow

สมมติว่านักศึกษาได้รับมอบหมายให้สร้างโมเดล Machine Learning เพื่อทำนายราคาที่ดินในจังหวัดขอนแก่น นักศึกษาจะนำ Large Language Models (เช่น ChatGPT, Claude) มาช่วยในขั้นตอน Data Preprocessing และ Model Evaluation อย่างไรให้เกิดประสิทธิภาพสูงสุด และมีข้อควรระวังใดที่ต้องคำนึงถึง

▸ แสดงแนวคำตอบ

ส่วนที่ 3: แบบฝึกหัด Coding

ข้อที่ 1: การสร้างโมเดล Supervised Learning (Classification) โจทย์: กำหนดให้ชุดข้อมูลตัวอย่างของแพลตฟอร์ม E-commerce แห่งหนึ่ง มีข้อมูลพฤติกรรมของลูกค้า ได้แก่ `เวลาที่ใช้งานบนเว็บไซต์ (นาที)` และ `จำนวนหน้าที่เข้าชม (หน้า)` รวมถึงตัวแปรเป้าหมายคือ `ซื้อสินค้า (1 = ซื้อ, 0 = ไม่ซื้อ)` ให้นักศึกษาเขียนโค้ด Python โดยใช้ไลบรารี `scikit-learn` เพื่อ:

1. แบ่งข้อมูลเป็นชุดฝึกสอน (Training Set) 80% และชุดทดสอบ (Testing Set) 20% โดยกำหนด `random_state=42`
2. สร้างและฝึกสอนโมเดล `LogisticRegression`
3. ทำนายผลลัพธ์จากชุดทดสอบ และคำนวณค่าความแม่นยำ (Accuracy Score)

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# ข้อมูลจำลอง (Synthetic Data)
data = {
    'time_spent_min': [5, 15, 25, 3, 10, 30, 40, 8, 20, 35],
    'pages_visited': [2, 5, 8, 1, 4, 10, 15, 3, 6, 12],
    'purchased': [0, 0, 1, 0, 0, 1, 1, 0, 1, 1]
}
df = pd.DataFrame(data)

# 1. กำหนด Features (X) และ Target (y)
X = df[['time_spent_min', 'pages_visited']]
y = df['purchased']

# แบ่งข้อมูล Training 80% และ Testing 20%
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. สร้างและฝึกสอนโมเดล (Instantiate and Fit)
model = LogisticRegression()
model.fit(X_train, y_train)

# 3. ทำนายผลและประเมินประสิทธิภาพ (Predict and Evaluate)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"ค่าความแม่นยำ (Accuracy) ของโมเดลคือ: {accuracy * 100:.2f}%")
# หมายเหตุ: เนื่องจากชุดข้อมูลมีขนาดเล็กมาก ค่า Accuracy อาจเป็น 100% หรือ 50% ขึ้นอยู่กับการสุ่ม
```

ผลลัพธ์:

```
ค่าความแม่นยำ (Accuracy) ของโมเดลคือ: 100.00%
```

ข้อที่ 2: การทำงานกับ Unsupervised Learning (Clustering) โจทย์: ร้านกาแฟบริเวณหลังมหาวิทยาลัยขอนแก่น ต้องการจัดกลุ่มลูกค้า (Customer Segmentation) เพื่อจัดทำโปรโมชัน โดยมีข้อมูลจำลองคือ `จำนวนครั้งที่มาเยือนใน 1 เดือน` และ `ยอดใช้จ่ายเฉลี่ยต่อครั้ง (บาท)` ให้นักศึกษาเขียนโค้ด Python เพื่อ:

1. จัดกลุ่มลูกค้าออกเป็น 3 กลุ่ม (Clusters) โดยใช้อัลกอริทึม `KMeans`
2. แสดงผลลัพธ์กลุ่มที่จัดได้ (Cluster Labels) เพิ่มเข้าไปเป็นคอลัมน์ใหม่ใน DataFrame
3. สร้างกราฟ Scatter Plot เพื่อแสดงผลการจัดกลุ่ม โดยใช้สีที่แตกต่างกันสำหรับแต่ละกลุ่ม

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# ข้อมูลจำลองพฤติกรรมลูกค้าร้านกาแฟ
data_coffee = {
    'visits_per_month': [2, 3, 4, 15, 18, 20, 8, 10, 9, 25, 22, 28],
    'avg_spend_thb': [45, 50, 40, 120, 150, 130, 200, 220, 250, 60, 55, 65]
}
df_coffee = pd.DataFrame(data_coffee)

# 1. สร้างและประยุกต์ใช้โมเดล KMeans (กำหนด k=3 ตามโจทย์)
# กำหนด n_init='auto' เพื่อหลีกเลี่ยง Warning ใน scikit-learn เวอร์ชันใหม่
kmeans = KMeans(n_clusters=3, random_state=42, n_init='auto')

# 2. ทำการจัดกลุ่มและเพิ่มคอลัมน์ Cluster ลงใน DataFrame
# ใช้ fit_predict เพื่อเรียนรู้และทำนายจัดกลุ่มในขั้นตอนเดียว
df_coffee['Cluster'] = kmeans.fit_predict(df_coffee)

print("ข้อมูลลูกค้าพร้อมผลการจัดกลุ่ม:")
print(df_coffee)
```

```python
# 3. สร้างกราฟ Scatter Plot เพื่อดูผลการจัดกลุ่ม (Data Visualization)
plt.figure(figsize=(8, 5))
scatter = plt.scatter(df_coffee['visits_per_month'],
                      df_coffee['avg_spend_thb'],
                      c=df_coffee['Cluster'],
                      cmap='viridis',
                      s=100)

plt.title('Customer Segmentation (Coffee Shop near KKU)')
plt.xlabel('Visits per Month (ครั้ง)')
plt.ylabel('Average Spend per Visit (THB)')
plt.colorbar(scatter, label='Cluster Label')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

ผลลัพธ์:

```
ข้อมูลลูกค้าพร้อมผลการจัดกลุ่ม:
   visits_per_month  avg_spend_thb  Cluster
0                 2             45        2
1                 3             50        2
2                 4             40        2
3                15            120        0
4                18            150        0
5                20            130        0
6                 8            200        1
7                10            220        1
8                 9            250        1
9                25             60        0
10               22             55        0
11               28             65        0
```

Scatter Plot: แสดงจุดข้อมูลลูกค้า 12 ราย แบ่งออกเป็น 3 กลุ่มด้วยสีที่แตกต่างกัน กลุ่มที่ 0 (มาบ่อย ยอดปานกลาง) กลุ่มที่ 1 (มาปานกลาง ยอดสูง) และกลุ่มที่ 2 (มาน้อย ยอดต่ำ) ทำให้ร้านกาแฟสามารถออกแบบโปรโมชันที่ตรงกลุ่มเป้าหมายได้

← บทที่ 6: สถิติเชิงพรรณนา EDA และ Data V

บทที่ 8: Regression — การทำนายค่าต่อเนื →
